1. INTRODUCTION

1.1 ABOUT PROJECT

Tech Shopway is an innovative and intelligent hardware discovery and project design platform that helps engineering students overcome difficulties in the development process of their projects through artificial intelligence technology. Completely shifting its approach from the conventional e-commerce method, which solely revolves around buying and selling products online, Tech Shopway has been built to be a digital ecosystem of information sources. Tech Shopway caters to the specific needs of engineering students to move from concept to reality by using hardware.

Fundamentally speaking, Tech Shopway serves as a dynamic aggregation and technical directory service. The platform offers the students a meticulously curated library of electronic parts, microcontrollers, and development boards that have direct connections with local suppliers. The platform facilitates users in conducting live price comparisons and sourcing the necessary components without making physical trips to various suppliers or searching through scattered internet forums. Moreover, Tech Shopway also features the “Kit Builder” utility. Through this utility, students can easily construct and store the required technology stack for future projects.

The distinguishing feature of Tech Shopway is the advanced implementation of Generative Artificial Intelligence. Through the use of Retrieval-Augmented Generation technology, the AI makes use of Google's Gemini and Groq Large Language Models powered by ChromaDB Vector Database for RAG implementation. The AI here operates like a technical mentor for the student. As the student constructs the kit, the AI evaluates the parts chosen, cross-checks the hardware data, and compiles an assessment on the feasibility of the project. The AI checks for logic and hardware compatibility, thus avoiding any unnecessary purchases by the student.

The project is built with a robust and decoupled architecture based on modern industry practices from a software engineering perspective. The SPA user interface for the project is built using React.js and Vite, providing a seamless user experience. The back-end logic and routing of RESTful APIs to provide integrations with Artificial Intelligence (AI) systems are built using Python and the Django REST Framework. The data for this project is persisted in a MySQL database and secured. In addition, in order to ensure maximum scalability and to eliminate issues with deploying the application to different environments, the entire technology stack is being containerized using docker and orchestrated, so that it can run smoothly on different types of operating systems.

1.2 OBJECTIVES OF PROJECT

1.2.1 Centralized Hardware Discovery and Vendor Aggregation:

The primary objective of this platform is to eliminate the fragmentation currently present in the hardware procurement process. Students typically waste valuable development time cross-referencing multiple generic online marketplaces or physically visiting local stores to find specific microcontrollers, sensors, and development boards. Tech Shopway aims to consolidate this process by acting as an intelligent, unified directory. The objective is to provide a single interface where students can browse specialized technical components and directly compare pricing and availability from local vendors, ensuring cost-effective and rapid procurement.

1.2.2 Virtual Architecture via the “Kit Builder” Ecosystem:

A major objective is to shift the educational paradigm from reactive purchasing to proactive project planning. To achieve this, the project aims to engineer an interactive “Kit Builder” module. The goal is to provide a digital sandbox where students can virtually architect their project’s technological stack before making any physical or financial commitments. By allowing users to mix and match components, save configurations, and manage their physical resources in a digital workspace, the platform aims to significantly reduce hardware waste and structural project errors. Also, they can be published on the platform by the user.

1.2.3 AI-Driven Feasibility Analysis and Technical Mentorship:

A groundbreaking objective of Tech Shopway is the integration of advanced Artificial Intelligence to act as a virtual technical mentor. Utilizing a Retrieval-Augmented Generation (RAG) architecture with Google's Gemini and Groq LLMs, the objective is to allow the platform to analyze the components within a student's "Kit Builder" and dynamically generate comprehensive feasibility reports. The AI Chatbot (powered by LLM) must be capable to understand context of your kits, and products (either you are vender or normal user), helping to identifying logic-level mismatches, evaluating the overall risk and viability of the proposed project, thereby bridging the gap between student ambition and technical reality. Not only Instructions, the chatbot should come with agentic features, like creating kits, publish them, find which vender should be good, etc. just via prompting.

1.2.4 Comprehensive Budgetary Tracking and Financial Optimization: Engineering projects, particularly at the undergraduate level, are often constrained by strict student budgets. A key objective of Tech Shopway is to provide integrated financial oversight during the planning phase. By aggregating real-time pricing data from various vendors (from online e-commerce to local shops) directly into the Kit Builder, the platform aims to dynamically calculate cumulative project costs. This objective ensures that students can optimize their component choices based on financial constraints, compare alternative hardware setups for cost efficiency, and prevent unexpected budget overruns before physical assembly begins.

1.2.5 Recommend best Vendor: LLM powered chatbot may help you to suggest best vendor for product, but to due to rate limits, its not always possible. So we will implement Topsis, an algorithm implemented to suggest best vendor for that product whenever you search for products for your project.

1.2.6 Platform to Review Published Kits, and Vendors: Platform should provide a way to post reviews, not only for published kits, but also for the vendors which are offering products in your region. So that not only users will know about the reputation of vendor, but also help Topsis to recommend best vendor for product during browsing, and chatbot to understand and suggest you best products.

2. SYSTEM ANALYSIS

2.1 INTRODUCTION

System analysis represents a critical, foundational phase in the software development life cycle (SDLC), serving as the definitive bridge between identifying an empirical real-world problem and designing its corresponding technical solution. In the context of software engineering, analysis involves a rigorous, methodical decomposition of current operational environments to understand their inherent limitations, bottlenecks, and data workflows. For this project, the system analysis phase was dedicated to evaluating the traditional, highly fragmented methodologies that engineering students currently utilize to conceptualize projects, verify hardware compatibility, and procure physical components.

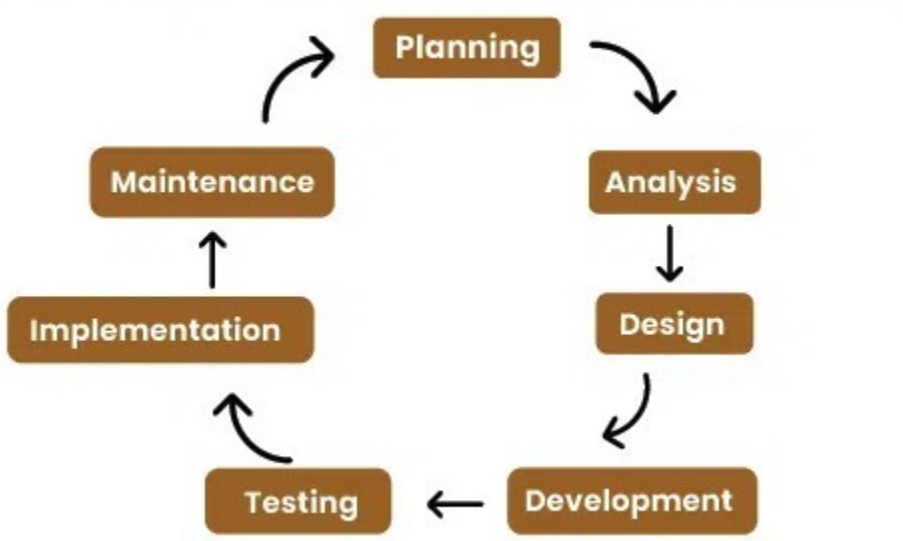

Image 2.1.1: SDLC

The primary objective of analyzing the existing ecosystem was to establish a definitive baseline. Currently, the "system" of student project development relies on decentralized manual research, spanning disparate vendor websites, static PDF datasheets, and generic online forums. By systematically mapping out this manual workflow, the development team was able to identify

critical points of friction, particularly the high probability of human error when matching microcontroller logic levels and the time-intensive nature of cross-referencing local vendor prices. This analytical phase provided the necessary empirical justification for shifting from a fragmented manual process to a centralized, AI-augmented digital ecosystem.

Furthermore, the system analysis phase for **Tech Shopway** extended beyond merely understanding the user's dilemma; it required a deep evaluation of the proposed technological integration. The team had to analyze how unstructured data (such as hardware documentation) could be logically processed by a machine. This involved evaluating the flow of data between a proposed React frontend, a Django backend, and an external Large Language Model (LLM). The analysis dictated that a simple relational database would be insufficient for complex hardware compatibility checks, thereby justifying the analytical requirement for a Retrieval-Augmented Generation (RAG) architecture and a ChromaDB vector database.

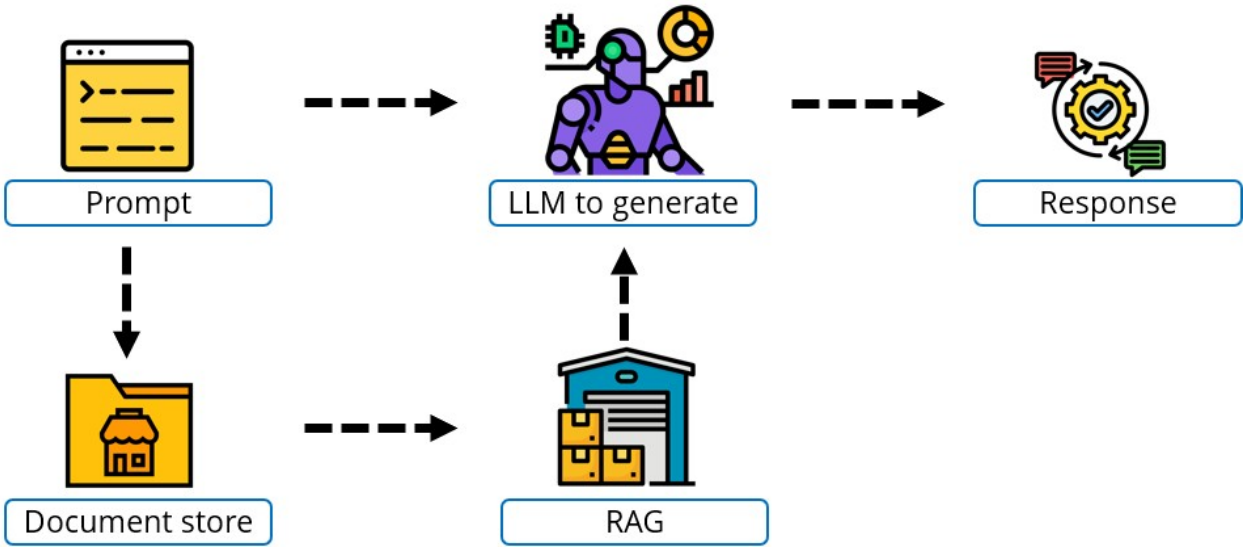

Image 2.1.2: RAG System

Ultimately, this chapter documents the comprehensive analytical journey of the project. It transitions from a broad identification of the core academic and logistical needs (Section 2.2), through a rigorous evaluation of the project’s technical and economic feasibility (Section 2.4), down to the granular mapping of data workflows using Data Flow Diagrams (Section 2.5). The insights derived from this system analysis directly inform the architectural blueprint, database schemas, and hardware requirements detailed in the subsequent system design phases, ensuring that Tech Shopway is engineered to solve the exact problems identified during this investigation.

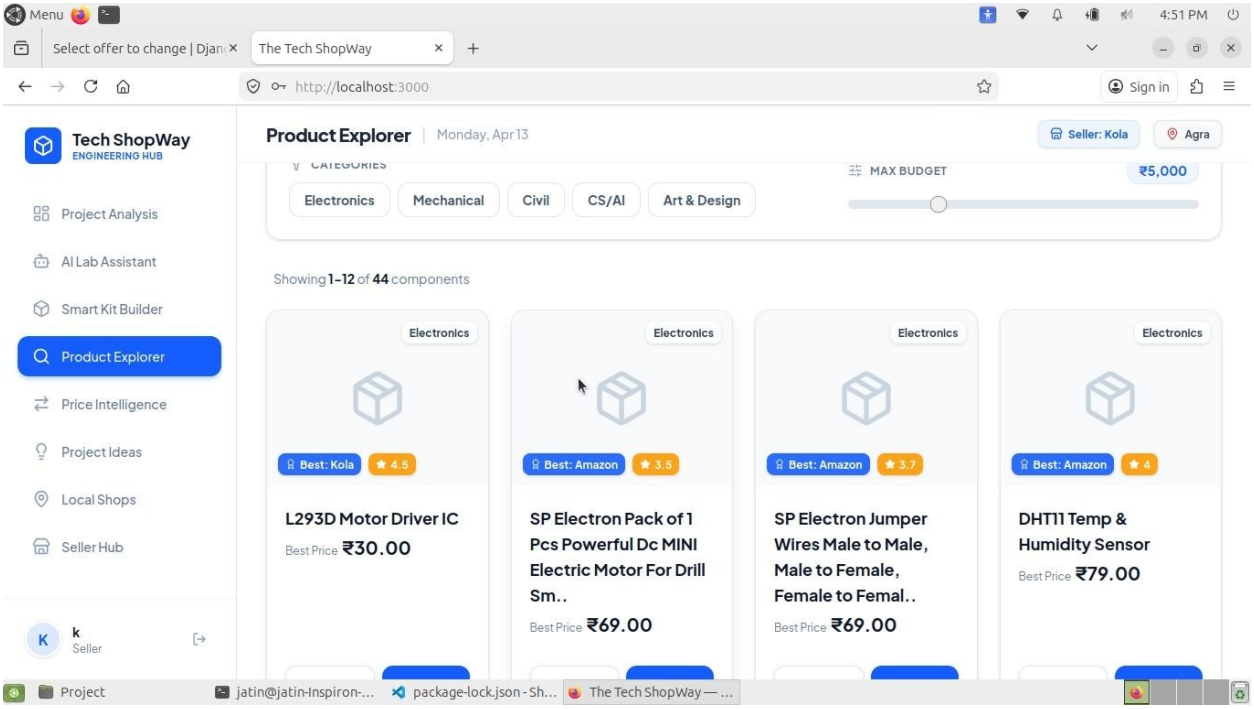

Image 2.1.3: Tech Shopway

2.2 IDENTIFICATION OF NEED

The need for "Tech Shopway" was identified through a survey of final-year engineering students. While building projects is a core part of an engineering degree, students often face several practical challenges when trying to get their ideas off the ground.

The fundamental need for this platform arises from the following pain points observed during our research:

2.2.1 **Fragmented Information (Hard to Find Parts):** There is currently no single platform dedicated to student technical supplies. General e-commerce sites (like Amazon) clutter search results with non-technical items, making it difficult to find specific electronics. On the other hand, local electronics shops have the parts but usually do not have a digital catalog. This forces students to waste time physically visiting multiple shops or navigating confusing general marketplaces.

2.2.2 **Incompatibility Issues (Parts Do Not Work Together):** Beginners frequently buy components that simply do not work together. For example, a student might buy a 3.3V sensor and try to connect it to a 5V microcontroller, which can cause the project to fail. Standard shopping websites just sell the items and do not warn users about these mismatches. Students need a smart system that flags these errors before they spend their money.

2.2.3 Budget Constraints (Overspending): Engineering students are usually on tight budgets. However, students often overspend by buying components from the first vendor they find, completely unaware that a cheaper option exists on another platform or at a nearby local shop. Because there is no easy way to compare prices in one place, students end up wasting money.

2.2.4 Lack of Guidance (Knowing What to Buy): Students often have great ideas and know exactly what they want to build (e.g., a "Drone" or an "Smart Weather Station"). However, they struggle to figure out the exact list of individual parts required to build it. There is a strong need for an intelligent guide or mentor that can take a project idea and tell the student exactly what hardware they need to buy.

2.2.5 Administrative Burden (Writing Project Reports): Before starting the physical work, colleges require students to submit formal project proposals to their guides. Manually researching and writing these hardware feasibility reports takes a lot of time away from the actual building process. Students need a tool that can take their selected parts and automatically generate a professional project report to submit to their teachers.

In short, the survey and analysis confirm that students do not just need another online store. They need a single, easy-to-use platform that helps them find the right parts, compares prices, warns them about hardware mistakes, and helps them generate their college reports.

2.3 PRELIMINARY INVESTIGATION

The preliminary investigation is the initial stage of the system analysis phase. Its purpose is to analyze how things are currently being done, identify the limitations of the current methods, and define the boundaries of the new proposed software. For this project, the investigation focused on how engineering students currently plan their hardware projects, verify technical details, and buy their components.

2.3.1 Study of the Existing System

Currently, the process of building a student project is highly manual, scattered, and time-consuming. When a student receives a project assignment, the existing workflow typically looks like this:

2.3.1.1 Manual Research: Students spend days reading online forums, watching tutorial videos, and asking seniors just to figure out what components they need.

2.3.1.2 Guesswork in Compatibility: To check if parts work together, students have to manually download and read complex PDF datasheets. Because they lack experience, they often guess incorrectly and buy incompatible parts. Even in the era of AI, students

often gets confused what is right and wrong, often due to hallucinating problem of AI, as AI often don’t have direct access to accurate and updated product info.

2.3.1.3 Scattered Resources: Students either visit multiple local electronics shops in person to compare prices, or they use generic websites like Amazon, which do not cater specifically to technical components. Similarly for chatbots, some students either use Gemini, or Claude.

2.3.1.4 Manual Budgeting: Students track their project costs using pen and paper or basic spreadsheets, which makes it hard to compare alternative setups.

The existing system relies entirely on the student’s own limited knowledge and takes focus away from actually writing code and building the project.

2.3.2 The Proposed System

Based on the flaws of the existing manual process, we proposed the development of "Tech Shopway." The proposed system aims to replace manual research with an automated, intelligent digital platform.

Instead of visiting multiple shops, the platform brings local vendor listings and component prices into one unified dashboard. Instead of guessing hardware compatibility, the proposed system introduces a "Kit Builder" backed by an AI assistant. This AI acts as a virtual guide, automatically checking the selected parts, warning the student about mismatches, and generating a formal feasibility report.

2.3.3 Scope of the Project

During the preliminary investigation, it is crucial to define what the system *will* do and what it *will not* do to keep the development realistic.

The scope includes:

2.3.3.1.1 Creating a digital catalog of electronics and microcontrollers for students.
2.3.3.1.2 Displaying price comparisons from various local and online vendors.
2.3.3.1.3 Providing an interactive "Kit Builder" for virtual project assembly.
2.3.3.1.4 Integrating a Generative AI chatbot to analyze the selected components and generate downloadable PDF project reports.
2.3.3.1.5 For vendors, help them to list them, so students know which vendors are available in their locality (city).

The scope does NOT include:

2.3.3.2.1 Managing physical warehouses or maintaining real-world hardware inventory.
2.3.3.2.2 Handling physical packaging, shipping, or delivery logistics.

2.3.3.2.3 Processing direct online financial transactions (payment gateways). The platform acts as a bridge and aggregator, not a direct seller.

By clearly defining this scope, the preliminary investigation ensured that the development team focused entirely on building an intelligent planning tool rather than getting bogged down by the logistics of a traditional e-commerce business.

2.4 FEASIBILITY STUDY

2.4.1 Introduction

A feasibility study is a critical checkpoint in the software development life cycle. Before writing any code, it is essential to evaluate whether the proposed system is practical, possible to build, and worth the investment of time and resources.

Conducted immediately following the preliminary investigation and requirement gathering phases, it serves as a rigorous analytical checkpoint before any actual programming or system design begins. The primary objective of this study is to critically evaluate whether the proposed software system is practically viable, objectively necessary, and worth the investment of time, effort, and computational resources. Rather than assuming a project will succeed simply because the initial concept is innovative, the feasibility study provides a structured framework to uncover potential development roadblocks, assess structural risks, and determine if the development team possesses the capability to deliver a functional, production-ready product. In essence, it answers the fundamental engineering question: "Can we build this system, and more importantly, should we build it?"

For "Tech Shopway," the feasibility study was conducted to ensure that building an AI-powered hardware aggregator and project-planning platform was realistic for a team of final-year engineering students. The project was evaluated across three primary dimensions:

2.4.1.1 Technical Feasibility: To measure the availability of necessary software tools and assess whether the development team possessed the required technical proficiency to implement a RAG-based AI pipeline.

2.4.1.2 Economic Feasibility: To ensure the project architecture could be fully realized without exceeding the strict, zero-capital budget constraints typical of academic student projects.

2.4.1.3 Operational Feasibility: To analyze whether the target end-users, specifically engineering students, would genuinely find the platform intuitive, easy to navigate, and beneficial to their project-building workflows.

2.4.2 Technical Feasibility

Technical feasibility evaluates whether the required hardware and software technologies exist to build the proposed system, and whether the development team possesses the necessary technical proficiency to implement them. For "Tech Shopway," the technical feasibility assessment was highly positive, as the architecture relies on well-documented, industry-standard frameworks and accessible cloud APIs. The evaluation was systematically broken down into application development, artificial intelligence integration, and system deployment.

2.4.2.1 System Specifications: Technical feasibility requires defining the physical hardware boundaries of the project. The platform is designed to be hardware-efficient. As we ourselves don’t posses extensive architecture to develop such big systems, we relied on only necessary tech stack for current system.

Table 2.4.2.1.1: System Untilized

Hardware Component	Specification Utilized
Processor (CPU)	Intel Core i5 M460
Memory (RAM)	4 GB
Storage	128 GB SSD
Graphics (GPU)	ATI Radeon HD 5000 Series
Operating System	Linux (Ubuntu MATE)
Browser	Firefox

2.4.2.2 Application Architecture and Frameworks: The platform is designed using a modern, decoupled architecture, separating the client-side interface from the server-side logic. The selected frameworks are open-source, highly stable, and well-supported by global developer communities.

Table 2.4.2.2.1: Applications Architecture

Architectural Component	Technology Selected	Feasibility Justification & Usage
Frontend	React.js (via Vite)	Highly feasible. Don't Require much hardware usage.
Backend	Python (Django REST)	Highly feasible. Django provides robust, built-in security, ORM, and native routing for complex AI API requests.
Relational Database	MySQL 8.0	Highly feasible.

2.4.2.3 AI Integration: A core component of Tech Shopway is its AI-driven feasibility analyzer. Implementing a Retrieval-Augmented Generation (RAG) pipeline is technically feasible due to the availability of robust, free-tier developer tools and APIs. As well as supports agentic behaviour without running LLM locally, saving costs and making it feasible for us to utilize them.

Table 2.4.2.3.A : AI System Used

AI System Component	Technology Selected	Feasibility Justification & Usage
Agentic Tasks	Gemini, Groq	Highly feasible. Offers an extensive Python SDK, allowing seamless integration with the Django backend to perform agentic tasks.

AI System Component	Technology Selected	Feasibility Justification & Usage
LLMs	Gemini, Groq	Highly feasible.
Vector Database	ChromaDB	Highly feasible. Runs locally and is optimized to store high-dimensional embeddings of complex hardware datasheets for context retrieval.

2.4.3 Economic Feasibility

Economic feasibility, often referred to as a cost-benefit analysis, is the process of evaluating the financial implications of a proposed system.

The economic feasibility of Tech Shopway is exceptionally high. The development strategy was explicitly designed to operate on a zero-capital budget by leveraging the modern open-source software paradigm and cloud-based "free-tier" infrastructures. The financial viability of the project is broken down into two primary phases: Development Costs and Operational Costs.

2.4.3.1 Development Costs: The cost of developing the platform is virtually zero due to the strategic selection of the technology stack. Traditional software development often requires expensive enterprise licenses for IDEs, databases, and server environments. However, Tech Shopway circumvents these financial barriers:

2.4.3.1.1 Zero Licensing Fees: The entire core architecture, including React.js (frontend), Python/Django (backend), MySQL (database) is 100% open-source and free for both academic and commercial use.

2.4.3.1.2 Development Hardware: As outlined in the technical feasibility section, the development does not require specialized, high-cost computing hardware (such as dedicated GPUs). The application was successfully built and tested on existing hardware.

2.4.3.2 Operational Economics: A major financial concern for modern AI-integrated applications is the cost of executing Large Language Model (LLM) queries. Tech Shopway manages these operational costs effectively:

2.4.3.3 Cloud API Utilization: Instead of training and hosting a proprietary AI model (which would cost thousands of dollars in cloud computing fees), the platform utilizes the developer APIs of Google Gemini and Groq. These platforms offer highly generous free

tiers that are more than sufficient for development, testing, and academic demonstration purposes. Not only that, students can bring their own keys to run these llms.

2.4.3.4 No Physical Inventory or Supply Chain Costs: Because Tech Shopway functions as an aggregator and a technical directory rather than a traditional e-commerce warehouse, the organization bears absolutely no costs related to purchasing physical electronic components, managing inventory space, or handling shipping logistics.

2.4.3.A Resource Economic Condition

Resource Requirement	Traditional Commercial Cost	Tech Shopway Financial Solution	Estimated Cost
Development Software	High (Enterprise IDEs/DBs)	Open-source stack (VS Code, React, Django, MySQL)	₹0.00
AI Inference Engine	High (Dedicated GPU Hosting)	Google Gemini & Groq API (Free Developer Tiers)	₹0.00
Vector Database	Medium (Cloud Vector DBs)	ChromaDB (Running locally)	₹0.00
Physical Hardware Inventory	Extremely High	Zero inventory. Platform acts strictly as an aggregator.	₹0.00
Deployment / Hosting	Low to Medium	Localhost for testing, or free-tier cloud platforms (e.g., Render, Vercel, Docker)	₹0.00

2.4.4 Operational Feasibility

Operational feasibility assesses whether the system will be successfully adopted by its intended users and how well it integrates into their daily workflows.

Tech Shopway is highly operationally feasible for the following reasons:

2.4.4.1 High User Adoption: The target audience (engineering students) is highly tech-savvy. The modern, React-based interface provides a familiar and intuitive experience, ensuring a minimal learning curve.

2.4.4.2 AI Analysis: Through a dedicated "Analyze" section and an integrated "AI Lab Assistant" chatbot, students can evaluate their finalized component kits. The AI utilizes LLM's global data to answer complex compatibility questions and verify hardware logic. *(Note: Real-time component validation, voltage indication, and live circuit diagram generation during the actual product selection phase are planned as future scope).*

2.4.4.3 Streamlined Academic Workflows: The platform can generate formal feasibility reports based on the user's finalized kits, significantly reducing the administrative burden of writing manually written project synopsis.

2.4.4.4 Centralized Ecosystem: It eliminates the manual, time-consuming process of visiting multiple local shops or generic websites by providing a single, dedicated aggregator for technical components and real-time pricing.

2.4.4.5 Low Administrative Overhead: Because the system acts as an aggregator rather than a traditional e-commerce store, it requires zero management of physical inventory, shipping logistics, or direct financial transactions, making it exceptionally easy to maintain.

2.4.4.A Operational Impact Assessment

Task	The Old Way (Manual)	The New Way	Main Benefit
Finding Parts	Searching through generic websites (like Amazon) or physically visiting multiple local shops.	Browsing one dedicated platform that lists technical parts and local vendor prices.	Saves time and makes finding specific components much easier.

Task	The Old Way (Manual)	The New Way	Main Benefit
Checking Compatibility	Guessing if parts work together or reading long, confusing PDF manuals.	Using the "Analyze" section or asking the "AI Lab Assistant" chatbot about the finalized kit.	Stops students from wasting money on parts that do not work together.
Managing the Budget	Adding up costs using pen and paper or basic Excel spreadsheets.	The Kit Builder automatically calculates the total cost of the selected items.	Helps students easily stay within their limited college budgets.
Writing College Reports	Typing out long hardware lists and project proposals manually for teachers.	Exporting automatically generated project feasibility reports directly from the platform.	Makes college paperwork much faster and less stressful.

2.5 DATAFLOW DIAGRAM

2.5.1 Introduction

A Data Flow Diagram (DFD) is a graphical representation of how data moves through an information system. Instead of focusing on the technical code or hardware, a DFD illustrates the logical flow of information—showing where data comes from, how it is processed, where it is stored, and where it ultimately goes. It is one of the most crucial tools in system analysis because it provides a clear, visual blueprint that both developers and non-technical stakeholders can easily understand.

For the **Tech Shopway** project, mapping the data flow is essential due to the decoupled architecture of the platform. The system handles multiple streams of data moving between the user's browser (React frontend), the server (Django backend), the database (MySQL), and external Artificial Intelligence services (Google Gemini).
A standard DFD uses four basic symbols to map this journey:

2.5.1 **External Entities**

- Shape **:** Squares
- Represent outside sources that interact with the system, such as the Student (User) or the Admin.

2.5.2 **Processes:**

- Shape **:** Circles or Rounded Rectangles
- Represent actions that transform data, such as "Add to Kit," "Calculate Total Cost," or "Analyze with AI."

2.5.3 **Data Stores:**

- Shape **:** Open-ended Rectangles
- Represent where data is saved for future use, such as the User Profile Database, Component Catalog, or Saved Kits (MySQL database).

2.5.4 **Data Flows:**

- Shape **:** Arrows
- Represent the actual path the data takes as it moves between entities, processes, and data stores.

To provide a comprehensive view, the data flow for Tech Shopway is broken down into multiple levels. It starts with a broad overview of the entire system (Level 0 / Context Diagram) and gradually zooms in to show the detailed internal workings of specific modules, such as the Kit Builder and the AI Lab Assistant (Level 1 and Level 2 DFDs).

2.5.2 0-Level DFD (Context Diagram)

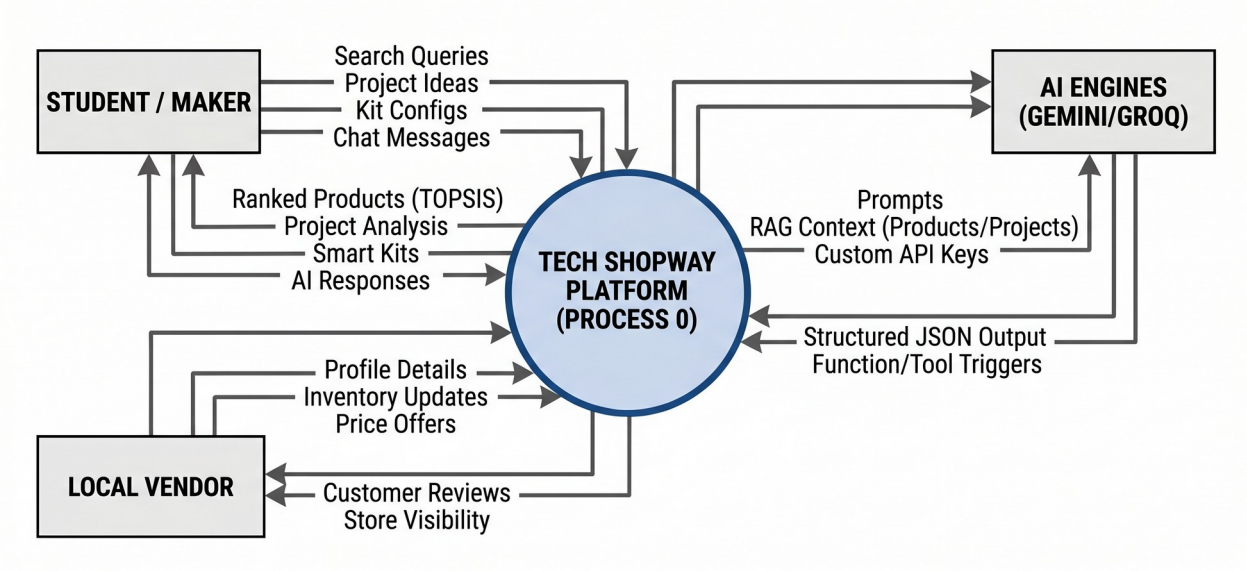

Image 2.5.2.1: 0 Level DFD

2.5.3 1-Level DFD

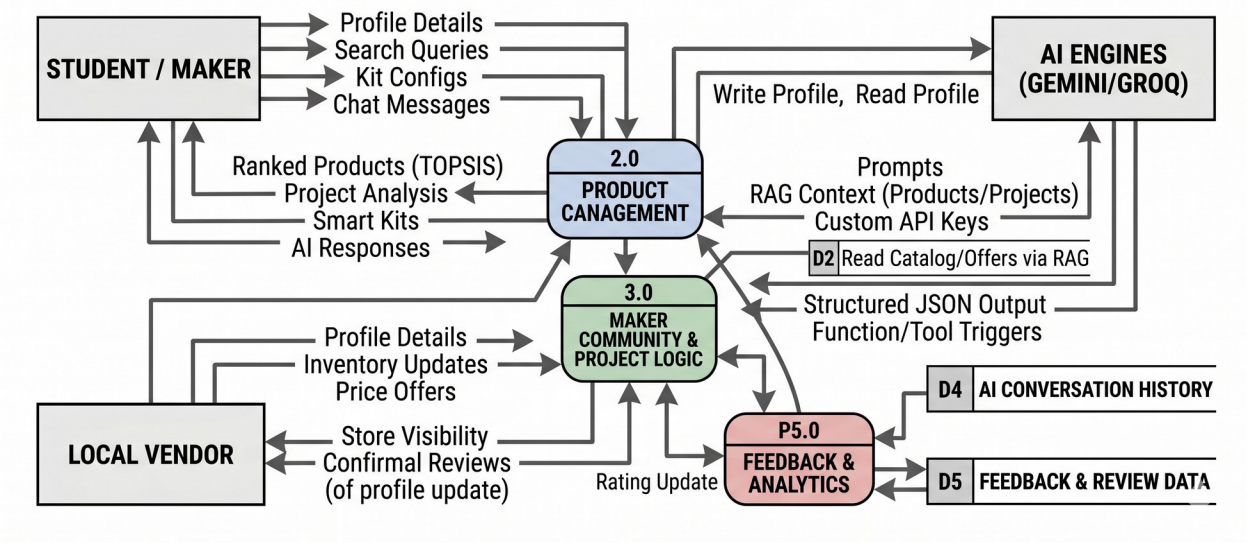

Image 2.5.3.1: 1 Level DFD

2.5.4 2-Level DFD

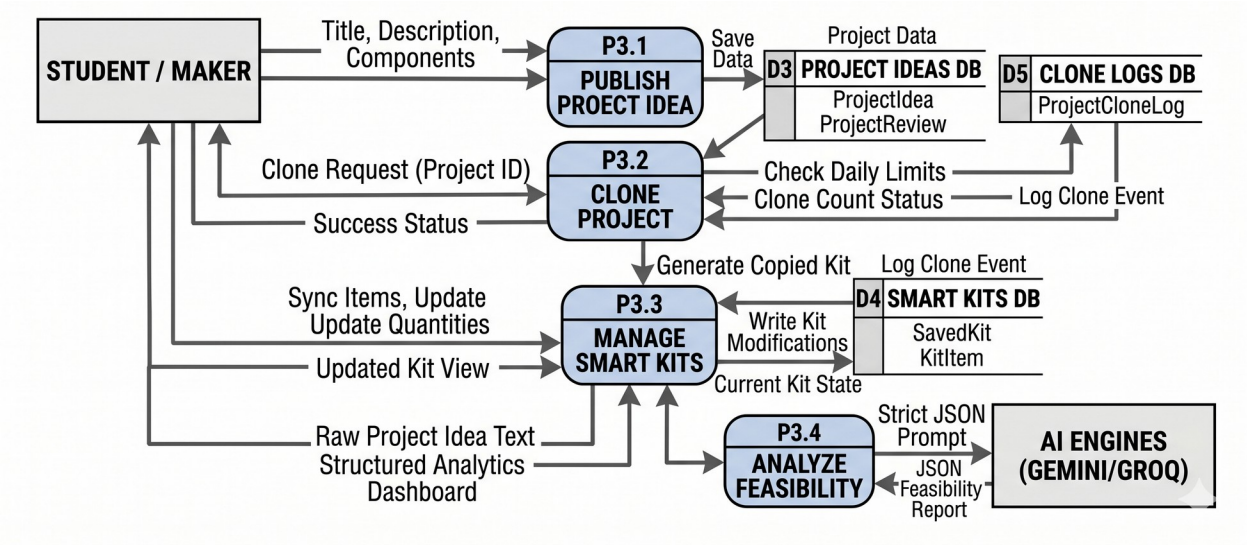

Image 2.5.4.1: 2 Level DFD

2.6 HARDWARE REQUIREMENTS

2.6.A Server Side (Developers)

Component	Minimum Specification	Recommended Specification (with running LLM locally)
Processor (CPU)	Intel Core i3 (5th Gen) / AMD Ryzen 3	Intel Core i7 / AMD Ryzen 5 or higher
Memory (RAM)	4 GB	16 GB
Storage	256 GB SSD	512 GB NVMe SSD
Graphics (GPU)	Integrated Graphics	NVidia Geforce RTX 2090 or higher.
Network	Broadband connection (for API calls)	High-speed stable connection

2.6.B Client Side (User)

Component	Minimum Specification	Recommended Specification
Processor (CPU)	Intel Dual Core	Intel Core i3 (5th Gen) / AMD Ryzen 3 or higher
Device Type	PC, Laptop, Tablet	PC, Laptop
Memory (RAM)	2 GB	4 GB or higher
Storage	No need (runs on browser)	No need (runs on browser)
Network	Stable Mobile Connection (1 Mbps)	High-speed stable connection

2.7 SOFTWARE REQUIREMENTS

2.7.1 For Client Side:

Table 2.7.1.A: Frontend

Language	HTML5, CSS3, JS
Framework	React (Vite), Tailwind CSS, Typescript

Table 2.7.1.B: Backend

Language	Python 3.10+
Framework	Django 5.0
Database	MySQL (SQL), ChromaDB (RAG)
Libraries	BeautifulSoup, google-generativeai, groq
API	RAPID, Gemini, Groq

Table 2.7.1.C: Other Tools

OS	Windows (64 bit), Linux (x86)
IDE	VS Code, Anaconda
Containerization	Docker
Browser	Any browser supporting HTML5, CSS3 & React (Firefox, Chrome, Edge)

2.7.2 For Client Side:

Table 2.7.2.A Minimum Software Requirements for Client

OS	Windows, Linux
Browser	Any browser supporting HTML5, CSS3 & React (Firefox, Chrome, Edge)

3. SYSTEM DESIGN

3.1 INTRODUCTION

System Design is the critical transitional phase in the software development life cycle (SDLC) where the logical specifications and system requirements identified during the analysis phase are translated into a concrete physical blueprint. While system analysis focuses on *what* the proposed system must do to solve the identified problem, system design focuses entirely on *how* the system will accomplish those goals technically. It bridges the gap between the problem domain and the eventual solution domain.

For the **Tech Shopway** project, the objective of this phase is to architect a highly scalable, modular platform capable of handling multi-vendor inventory, real-time user interactions, and complex Artificial Intelligence processing.

The structural design of the platform is guided by a few core, practical principles:

3.1.1 Decoupled Architecture: Strictly separating the client-side user interface (React.js) from the server-side business logic (Django) via RESTful APIs.

3.1.2 Containerization: Utilizing Docker to encapsulate the database and backend services for secure, isolated deployment.

3.1.3 AI Integration: The design creates a clear, lightweight pathway to send a student's virtual project kit to the external AI engine (Gemini, Groq) so it can quickly return compatibility advice without slowing down the main website. As well as adding some other agent tasks like modifying, fetching client side data through prompting.

3.1.4 Optimized Storage: Implementing a dual-database approach using MySQL for relational user/vendor data and ChromaDB for rapid AI document retrieval.

3.1.5 Scalability: The foundation is built so that if the platform grows to include more colleges or more local shops, the system will not crash under the increased load.

3.2 ER DIAGRAM

3.2.1 Introduction

An Entity-Relationship (ER) Diagram is a visual blueprint of a system's database. While Data Flow Diagrams (DFDs) show how information moves through the software, an ER Diagram shows exactly how that information is structured, stored, and connected behind the scenes. It is a crucial step in system design because a poorly organized database will lead to slow search times, duplicate data, and system crashes.

For the **Tech Shopway** platform, managing data efficiently is highly complex. The system acts as a central hub connecting multiple different groups. It must keep track of who is using the site (Students), who is selling the parts (Local Vendors), what exactly is being sold (Technical Components), and the virtual hardware configurations the users are building (Saved Project Kits).

To map out the relational database for this project, the ER Diagram uses three standard visual symbols:

3.2.1.1 Entities (Rectangles): These represent the main "objects" or tables in the database. For Tech Shopway, primary entities include the Student, Local Vendor, Component, and Project Kit.

3.2.1.2 Attributes (Ovals): These represent the specific details stored inside an entity. For example, the Student entity will have attributes like Name, College Email, and Password.

3.2.1.3 Relationships (Diamonds): These illustrate how different entities interact with one another. For example, a Student "Builds" a Project Kit, and a Local Vendor "Supplies" a Component.

3.2.2 ER Diagrams of Project

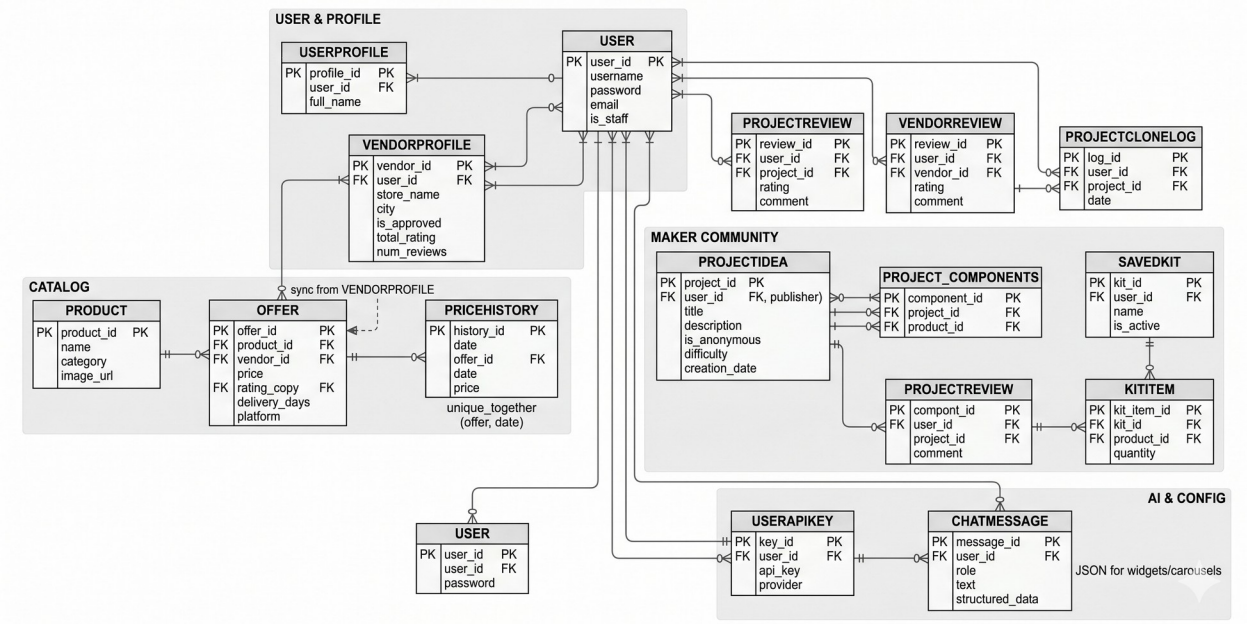

Image 3.2.2.1: ER Diagram

3.2.2.1 Core Database Entities & Attributes

These are the primary tables that store the foundational data for the platform.

3.2.2.1.1 User (Django Native)

Attributes: id, username, password, email

3.2.2.1.2 UserProfile

Attributes: user (FK), location

3.2.2.1.3 VendorProfile

Attributes: user (FK), vendor_name, location, address, website_url, description, phone_number, support_email, is_approved

3.2.2.1.4 Product

Attributes: id, name, category

3.2.2.1.5 Offer

Attributes: product (FK), vendor, vendor_type, vendor_location, price, stock, rating, delivery_days, url

3.2.2.1.6 PriceHistory

Attributes: product (FK), vendor, price, recorded_date

3.2.2.1.7 ProjectIdea

Attributes: id, student_name, title, description, clones

3.2.2.1.8 ProjectReview

Attributes: project (FK), user (FK), rating, comment, created_at

3.2.2.1.9 VendorReview

Attributes: vendor (FK), user (FK), rating, comment

3.2.2.1.10 ProjectCloneLog

Attributes: user (FK), project (FK), cloned_at

3.2.2.1.11 SavedKit

Attributes: user (FK), kit_name, description, created_at

3.2.2.1.12 ChatSession

Attributes: user (FK), title, updated_at

3.2.2.1.13 ChatMessage

Attributes: session (FK), role, text, created_at, structured_data (JSON)

3.2.2.1.14 UserAPIKey

Attributes: user (FK), gemini_key, groq_key

3.2.2.2 Junction Entities

These tables exist specifically to resolve Many-to-Many (M:N) relationships, often storing extra contextual data about the connection.

3.2.2.2.1 KitItem (Explicit Junction)

Purpose: Connects a SavedKit to multiple Products.

Attributes: kit (FK), product (FK), quantity

Why it's a junction: A kit contains many products, and a product can be in many kits. This table stores *how many* of a specific product are in a specific kit via the quantity field.

3.2.2.2.2 Project_Components (Implicit Junction)

Purpose: Connects a ProjectIdea to multiple Products.

Attributes: projectidea_id (FK), product_id (FK)

Why it's a junction: Handled automatically by Django's ManyToManyField on the components field in the ProjectIdea model. It maps which hardware parts are required to build a specific student project.

3.2.2.3 Entity Relationships and Cardinality

This outlines how the entities interact with one another and the numerical rules governing those connections.

One-to-One (1:1) Relationships

- **User ↔ UserProfile:** One user account has exactly one standard student profile.
- **User ↔ VendorProfile:** One user account can have exactly one vendor storefront profile.
- **User ↔ UserAPIKey:** One user has one designated vault for their custom LLM API keys.

One-to-Many (1:N) Relationships

- **Product ↔ Offer:** One product in the master catalog can have *many* offers from different local and online vendors.
- **Product ↔ PriceHistory:** One product has *many* historical price logs recorded over time.
- **VendorProfile ↔ VendorReview:** One vendor can receive *many* reviews from different users.
- **ProjectIdea ↔ ProjectReview:** One published project can receive *many* reviews/ratings.
- **User ↔ SavedKit:** One student can create and manage *many* different Smart Kits (workspaces).
- **User ↔ ChatSession:** One user can initiate *many* different chat threads with the AI assistant.
- **ChatSession ↔ ChatMessage:** One chat session contains *many* individual messages.

Many-to-Many (M:N) Relationships

- **SavedKit ↔ Product:** A kit contains *many* products, and a specific product can appear in *many* different students' kits. (Resolved via the KitItem table).
- **ProjectIdea ↔ Product:** A project requires *many* hardware components, and a single component (like an Arduino) is used in *many* different projects. (Resolved via the implicit components table)

3.3 DATA/TABLE STRUCTURES

Table 3.3.1 Chat Session

Field Name	Data Type	Key / Constraint	Description / Purpose
id	char(32)	**Primary Key** (PRI), NOT NULL	Unique 32-character identifier for the chat session.
title	varchar(255)	NOT NULL	Auto-generated or user-defined title for the conversation (e.g., "ESP32 Feasibility Check").
created_at	datetime(6)	NOT NULL	Timestamp recording when the session was initiated.
updated_at	datetime(6)	NOT NULL	Timestamp of the most recent interaction in the session.
user_id	int	**Foreign Key**	Links the chat session to the specific

Field Name	Data Type	Key / Constraint	Description / Purpose
		(MUL), NOT NULL	student (user) account.

Table 3.3.2 Chat Message

Field Name	Data Type	Key / Constraint	Description / Purpose
id	bigint	**Primary Key** (PRI), Auto Increment	Unique identifier for the individual message.
role	varchar(10)	NOT NULL	Identifies the sender (e.g., 'user', 'ai', 'system').
text	longtext	Nullable (YES)	The raw text content of the message or prompt.
structured_data	json	Nullable (YES)	Stores formatted data returned by the AI, such as a parsed Bill of Materials (BOM) or compatibility matrix.
created_at	datetime(6)	NOT NULL	Timestamp of when the message was sent.
session_id	char(32)	**Foreign Key**	Links the message back to its parent

Field Name	Data Type	Key / Constraint	Description / Purpose
		(MUL), NOT NULL	myapp_chatsession.

Table 3.3.3 Saved Kit

Field Name	Data Type	Key / Constraint	Description / Purpose
id	bigint	**Primary Key** (PRI), Auto Increment	Unique identifier for the project kit.
kit_name	varchar(100)	NOT NULL	The name given to the project by the student (e.g., "IoT Weather Station").
created_at	datetime(6)	NOT NULL	Timestamp of when the kit was created.
user_id	int	**Foreign Key** (MUL), NOT NULL	Links the kit to the student who owns it.
description	longtext	Nullable (YES)	Optional notes or project requirements written by the student.

Table 3.3.4 Kit Item

Field Name	Data Type	Key / Constraint	Description / Purpose
id	bigint	**Primary Key** (PRI), Auto Increment	Unique identifier for the line item.
quantity	int unsigned	NOT NULL	The number of units required for this specific component in the kit.
product_id	bigint	**Foreign Key** (MUL), NOT NULL	Links to the master component/product catalog.
kit_id	bigint	**Foreign Key** (MUL), NOT NULL	Links the component to the specific myapp_savedkit.

Table 3.3.5 Offer

Field Name	Data Type	Key / Constraint	Description / Purpose
id	bigint	**Primary Key** (PRI), Auto Increment	Unique identifier for the vendor's specific listing.
vendor	varchar(50)	NOT NULL	The name of the local shop or vendor providing the component.

Field Name	Data Type	Key / Constraint	Description / Purpose
price	decimal(10,2)	NOT NULL	The specific price set by this vendor.
rating	double	NOT NULL	The vendor's reliability or product quality rating.
delivery_days	int	NOT NULL	Estimated time to procure or deliver the item.
url	varchar(200)	NOT NULL	External link or internal routing path to the vendor's specific product page.
product_id	bigint	**Foreign Key** (MUL), NOT NULL	Links this offer to the master component catalog.
stock	int	NOT NULL	Current inventory count available at this vendor.
vendor_location	varchar(255)	Nullable (YES)	Physical address or region of the local shop.
vendor_type	varchar(20)	NOT NULL	Classification of the seller (e.g., 'local_shop', 'online_retailer').

Table 3.3.6 Product

Field Name	Data Type	Key / Constraint	Description / Purpose
id	bigint	**Primary Key** (PRI), Auto Increment	Unique identifier for the core hardware component.
name	varchar(255)	NOT NULL	The standardized name of the component.
category	varchar(50)	NOT NULL	Classification (e.g., Microcontroller, Sensor, Actuator).
tags	longtext	NOT NULL	Searchable keywords related to the component.
image_url	varchar(500)	Nullable (YES)	Link to the standardized image of the component for the UI.

Table 3.3.7 Price History

Field Name	Data Type	Key / Constraint	Description / Purpose
id	bigint	**Primary Key** (PRI), Auto Increment	Unique identifier for the historical record.
vendor	varchar(100)	NOT NULL	The name of the vendor

Field Name	Data Type	Key / Constraint	Description / Purpose
			associated with the historical price.
price	decimal(10,2)	NOT NULL	The recorded price of the component on that specific date.
recorded_date	date	NOT NULL	The exact date the price was logged into the system.
product_id	bigint	NOT NULL	Links the price record back to the core myapp_product.

Table 3.3.8 Project Idea

Field Name	Data Type	Key / Constraint	Description / Purpose
id	bigint	**Primary Key** (PRI), Auto Increment	Unique identifier for the published project idea.
title	varchar(200)	NOT NULL	The public title of the project (e.g., "Smart Home Automation").
description	longtext	NOT NULL	Detailed documentation, methodology, or abstract of the project.

Field Name	Data Type	Key / Constraint	Description / Purpose
difficulty	varchar(20)	NOT NULL	Categorization of complexity (e.g., Beginner, Intermediate, Advanced).
tags	varchar(200)	NOT NULL	Searchable keywords for easy discovery by other students.
category	varchar(100)	NOT NULL	Broad engineering domain (e.g., IoT, Robotics, Signal Processing).
clones	int	NOT NULL	A counter tracking how many times the project has been copied by others.
image_url	varchar(500)	Nullable (YES)	Cover image or schematic snapshot of the final project.
student_name	varchar(100)	NOT NULL	The author/creator of the project for academic attribution.
university	varchar(200)	NOT NULL	The institution where the project was developed.

Table 3.3.9 Project Idea Components

Field Name	Data Type	Key / Constraint	Description / Purpose
id	bigint	**Primary Key** (PRI), Auto Increment	Unique identifier for the specific component line item.
projectidea_id	bigint	**Foreign Key** (MUL), NOT NULL	Links to the parent published myapp_projectidea.
product_id	bigint	**Foreign Key** (MUL), NOT NULL	Links to the master myapp_product required for the build.

Table 3.3.10 Project Clone Log

Field Name	Data Type	Key / Constraint	Description / Purpose
id	bigint	**Primary Key** (PRI), Auto Increment	Unique identifier for the clone event log.
cloned_at	datetime(6)	NOT NULL	Timestamp of when the student copied the project.

Field Name	Data Type	Key / Constraint	Description / Purpose
project_id	bigint	**Foreign Key** (MUL), NOT NULL	Identifies which published project was cloned.
user_id	int	**Foreign Key** (MUL), NOT NULL	Identifies the specific student account that initiated the clone.

Table 3.3.11 User API Key

Field Name	Data Type	Key / Constraint	Description / Purpose
id	bigint	**Primary Key** (PRI), Auto Increment	Unique identifier for the credential record.
gemini_key	varchar(255)	Nullable (YES)	The encrypted Google Gemini API key provided by the user.
groq_key	varchar(255)	Nullable (YES)	The encrypted Groq API key for high-speed hardware inference.
user_id	int	**Unique Key** (UNI), NOT NULL	Strictly links the API keys to one specific user account (1:1 relationship).

Table 3.3.12 Project Review

Field Name	Data Type	Key / Constraint	Description / Purpose
id	bigint	**Primary Key** (PRI), Auto Increment	Unique identifier for the review.
rating	int	NOT NULL	A numerical score (e.g., 1 to 5 stars) given to the published project.
comment	longtext	NOT NULL	The written feedback, troubleshooting tips, or critique from the reviewer.
created_at	datetime(6)	NOT NULL	Timestamp of when the review was submitted.
project_id	bigint	**Foreign Key** (MUL), NOT NULL	Links the review to the specific published myapp_projectidea.
user_id	int	**Foreign Key** (MUL), NOT NULL	Identifies the student who wrote the review.

Table 3.3.13 Vendor Review

Field Name	Data Type	Key / Constraint	Description / Purpose
id	bigint	**Primary Key** (PRI), Auto Increment	Unique identifier for the vendor review.
rating	int	NOT NULL	A numerical score (e.g., 1 to 5 stars) given to the local shop.
comment	longtext	NOT NULL	The student's written experience regarding their interaction with the vendor.
created_at	datetime(6)	NOT NULL	Timestamp of when the review was posted.
user_id	int	**Foreign Key** (MUL), NOT NULL	Identifies the student who submitted the review.
vendor_id	bigint	**Foreign Key** (MUL), NOT NULL	Links the review directly to the specific myapp_vendorprofile.

Table 3.3.14 Vendor Profile

Field Name	Data Type	Key / Constraint	Description / Purpose
id	bigint	**Primary Key** (PRI), Auto Increment	Unique identifier for the vendor business profile.
vendor_name	varchar(100)	**Unique Key** (UNI), NOT NULL	The official business or shop name.
is_approved	tinyint(1)	NOT NULL	Boolean flag (True/False) controlled by the System Admin to authorize the vendor to sell.
user_id	int	**Unique Key** (UNI), NOT NULL	Links the business profile to the shop owner's base user account (1:1 relationship).
location	varchar(255)	Nullable (YES)	A generalized string or coordinate mapping for geographic search.
website_url	varchar(500)	Nullable (YES)	External link to the vendor's primary website, if they have one.
description	longtext	Nullable (YES)	A brief bio or description of the shop's specialties (e.g., "Specializing in IoT sensors").
phone_number	varchar(20)	Nullable (YES)	Primary business contact number for students to call.

Field Name	Data Type	Key / Constraint	Description / Purpose
support_email	varchar(254)	Nullable (YES)	Dedicated email for order or stock inquiries.
address	longtext	Nullable (YES)	The full physical street address of the local shop.

Table 3.3.15 Django Authentication for User Profile

Field Name	Data Type	Key / Constraint	Description / Purpose
id	int	**Primary Key** (PRI), Auto Increment	Unique internal identifier for the user account.
password	varchar(128)	NOT NULL	Securely hashed password (never stored as plaintext) input by the user.
last_login	datetime(6)	Nullable (YES)	Automatically records the timestamp of the user's most recent login.
is_superuser	tinyint(1)	NOT NULL	Boolean flag granting absolute administrative permissions.

Field Name	Data Type	Key / Constraint	Description / Purpose
username	varchar(150)	**Unique Key** (UNI), NOT NULL	The unique handle chosen by the student or vendor during registration.
first_name	varchar(150)	NOT NULL	User's first name (can be left blank internally if not required on UI).
last_name	varchar(150)	NOT NULL	User's last name (can be left blank internally).
email	varchar(254)	NOT NULL	The user's contact or college email address input during registration.
is_staff	tinyint(1)	NOT NULL	Boolean flag designating if the user can access the Django admin panel.
is_active	tinyint(1)	NOT NULL	Boolean flag controlling whether the account is currently enabled or banned.
date_joined	datetime(6)	NOT NULL	Automatically generated timestamp of when the account was created.

Table 3.3.16 Extended User Profile

Field Name	Data Type	Key / Constraint	Description / Purpose
id	bigint	**Primary Key** (PRI), Auto Increment	Unique identifier for the extended profile record.
location	varchar(255)	Nullable (YES)	The physical location input by the user during registration (used to match students with the closest local vendors).
user_id	int	**Unique Key** (UNI), NOT NULL	**Foreign Key** establishing a strict 1:1 relationship with auth_user.id.

4. SYSTEM IMPLEMENTATION AND MAINTENANCE

4.1 Introduction

The implementation phase marks the transition from theoretical system design to a functional software product. It involves writing the code, setting up the databases, configuring the servers, and integrating external APIs to build the Tech Shopway platform. Following successful implementation, system maintenance ensures the software continues to operate efficiently, securely, and accurately over its lifecycle, particularly as vendor catalogs expand and AI models evolve.

4.2 System Implementation

The construction of Tech Shopway utilizes a decoupled architecture, requiring separate implementation strategies for the backend, frontend, and the AI integration pipelines.

4.2.1 Environment Setup and Containerization

The foundation of the implementation relies on Docker to ensure consistency across development and deployment environments.

4.2.1.1 Docker Orchestration: `docker-compose` is utilized to spin up isolated containers for the Django web server, the MySQL relational database, and the ChromaDB vector store. This eliminates environment discrepancies and allows for rapid deployment.

4.2.1.2 Database Migration: The Entity-Relationship schema defined in the design phase is translated into Django models. Django's ORM (Object-Relational Mapper) is then used to generate and execute SQL migrations, constructing the tables for Students, Local Vendors, Components, and the necessary junction tables in the MySQL container.

4.2.2 Backend and AI Pipeline Implementation

The core business logic and API routing are handled by the server-side architecture.

4.2.2.1 Django REST Framework (DRF): Implemented to build secure, stateless API endpoints that handle user authentication, vendor inventory updates, and project kit modifications.

4.2.2.2 RAG Pipeline Construction: A Retrieval-Augmented Generation (RAG) architecture is implemented for the AI Lab Assistant. When a student requests a feasibility check on a saved kit, the backend triggers a process that queries the local

ChromaDB for the relevant component datasheets (using vector embeddings) and packages this context alongside the student's hardware list.

4.2.2.3 External API Integration: The packaged context is transmitted to the Google Gemini API (or Groq for high-speed inference) to generate the final compatibility report and budget optimization suggestions, which are then passed back to the frontend.

4.2.3 Frontend Implementation

The client-side interface is engineered to provide a responsive, real-time experience for both students and vendors.

- **React & Vite Configuration:** The frontend is built as a Single Page Application (SPA) using React.js. Vite is used for rapid module bundling, and TypeScript is enforced to catch data type errors during the build process.
- **State Management:** As students interact with the "Kit Builder" (adding or removing resistors, microcontrollers, etc.), client-side state management ensures the total estimated cost and power requirements update instantly without requiring server polling.

4.3 System Maintenance

To ensure the long-term viability of the multi-vendor aggregator, a structured maintenance protocol is established. Software maintenance for Tech Shopway is categorized into four primary types:

4.3.1 Corrective Maintenance

Focuses on identifying and resolving residual bugs or errors that emerge post-deployment.

4.3.1.1 Fixing UI rendering issues on different mobile browsers.

4.3.1.2 Resolving failed API calls or timeouts when the external Gemini/Groq services experience high latency.

4.3.1.3 Addressing routing errors in the multi-vendor catalog when local shops delete legacy components.

4.3.2 Adaptive Maintenance

Involves modifying the software to adapt to changes in the external environment or underlying infrastructure.

4.3.2.1 Updating the Django and React frameworks to their latest stable versions to patch security vulnerabilities.

4.3.2.2 Modifying the backend authentication logic if the college updates its email domain protocols.

4.3.2.3 Adjusting the AI prompt engineering if Google updates the underlying Gemini model architecture, ensuring the output format remains consistent for the student dashboard.

4.3.3 Perfective Maintenance

Enhancing the system to improve performance, maintainability, or add new features requested by users.

4.3.3.1 Optimizing MySQL query indexing to speed up search results as the component catalog grows to thousands of parts.
4.3.3.2 Enhancing the AI's capability from simply checking hardware compatibility to generating base-level Python/C++ starter code for the student's specific microcontroller kit.
4.3.3.3 Adding bulk-upload functionality (e.g., CSV parsing) for local vendors to update their entire inventory pricing in a single action.

4.3.4 Preventive Maintenance

Proactive measures taken to prevent future system failures or data loss.

4.3.4.1 Implementing automated daily backups of both the MySQL relational database and the ChromaDB vector store.
4.3.4.2 Regularly monitoring external API quota limits and billing alerts to prevent sudden service outages for the AI Lab Assistant feature.
4.3.4.3 Periodically refreshing and re-embedding the technical datasheets in the vector database to ensure the AI has access to the most current hardware specifications and warning protocols.

5. SYSTEM TESTING

5.1 INTRODUCTION

System Testing is a critical phase in the Software Development Life Cycle (SDLC) that ensures the developed application meets all specified technical, functional, and business requirements. After the implementation phase is complete, testing acts as the final quality assurance checkpoint before the system is deployed to end-users. The primary objective is to identify, document, and resolve defects (bugs), ensuring that the software is reliable, secure, and performs optimally under various conditions.

For the **Tech Shopway** platform, testing is particularly vital due to its complex, decoupled architecture and multi-actor environment. The system must flawlessly handle concurrent interactions from Students, Local Vendors, and the System Admin, while also managing external asynchronous calls to the Artificial Intelligence APIs.
The testing strategy for Tech Shopway focuses on verifying three main pillars of the platform:

5.1.1 Data Integrity: Ensuring that vendor prices update correctly, student kits save accurately, and the junction tables (handling the many-to-many relationships) do not create duplicate or orphaned records in the MySQL database.

5.1.2 AI Reliability: Validating that the Retrieval-Augmented Generation (RAG) pipeline successfully pulls the correct datasheets from ChromaDB and that the external Gemini API returns properly formatted, accurate compatibility reports without timing out.

5.1.3 UI/UX Consistency: Guaranteeing that the React.js frontend updates instantly, responds correctly on different screen sizes, and handles errors gracefully (e.g., notifying a user if a component is suddenly out of stock).

To achieve a robust evaluation, the following sections in this chapter will detail the specific testing methodologies applied to the system, including **Unit Testing** (testing individual code modules), **Integration Testing** (verifying how the backend, frontend, and databases interact), **Functional Testing** (testing specific features like the Kit Builder), and **User Acceptance Testing (UAT)**.

5.2 Testing Techniques

To guarantee the reliability and stability of the Tech Shopway platform, a multi-tiered testing strategy was employed. Because the system utilizes a decoupled architecture, separating the React.js frontend, the Django backend, and the external AI APIs, testing must occur at individual module levels as well as across the integrated system as a whole.

The following testing techniques were utilized during the development lifecycle:

5.2.1 Unit Testing: Unit testing focuses on verifying the smallest testable parts of the application in isolation to ensure each component functions exactly as designed.

5.2.1.1 Backend: Individual Python functions and database models were tested. For example, testing the logic that calculates the total cost of a project kit to ensure it accurately aggregates the localized prices from the `VENDOR_INVENTORY` junction table without mathematical errors.

5.2.1.2 Frontend: UI components were tested in isolation. For instance, verifying that the "Add to Kit" button correctly updates the localized state of the Kit Builder interface before sending the payload to the server.

5.2.1.3 AI Pipeline: Testing the specific Python function responsible for embedding a search query and retrieving the correct text chunk from the local ChromaDB, completely independent of the external Gemini API.

5.2.2 Integration Testing: Once individual units were validated, integration testing was conducted to verify that different modules communicate and transfer data correctly.

5.2.2.1 Database Integration: Ensuring the Django ORM successfully performs complex CRUD (Create, Read, Update, Delete) operations across both the relational MySQL database and the ChromaDB vector store simultaneously without data mismatch.

5.2.2.2 API Endpoint Testing: Verifying that the RESTful API routes correctly accept JSON payloads from the frontend and return the appropriate HTTP status codes (e.g., returning a `200 OK` when a vendor updates a price, or a `401 Unauthorized` if a student tries to access the vendor dashboard).

5.2.2.3 External Service Integration: Critically testing the connection to the Google Gemini/Groq APIs, ensuring that context prompts are securely transmitted and that the application handles API timeouts or rate-limit errors gracefully.

5.2.3 Functional and System Testing: Functional testing evaluates the system against the predefined business requirements from the end-user's perspective. It tests complete workflows from start to finish.

5.2.3.1 Multi-Vendor Workflow: Testing the scenario where a local vendor logs in, adds a new microcontroller to their inventory, sets a localized price, and verifying that this new price immediately reflects in the global component catalog for students.

5.2.3.2 Kit Builder Workflow: Testing a student's ability to search for components, add them to a virtual kit, request an AI feasibility analysis, and successfully download the final optimized Bill of Materials (BOM) and report.

5.2.4 Security and Authorization Testing: Given the multi-actor nature of the platform, strict security boundaries must be enforced.

5.2.4.1 Role-Based Access Control (RBAC): Verifying that session tokens correctly identify the user role. A Student must never be able to access the Vendor dashboard, and a Vendor must only be able to modify their *own* inventory pricing, not the pricing of competitor shops.

5.2.4.2 Data Validation: Testing input fields to prevent common web vulnerabilities like SQL Injection or Cross-Site Scripting (XSS), particularly in areas where vendors can input text descriptions for components.

5.2.5 Performance and Load Testing: Performance testing ensures the system remains responsive under expected operational stress.

5.2.5.1 Concurrent API Requests: Simulating multiple students requesting AI feasibility reports simultaneously to monitor how the Django backend handles asynchronous external API calls without blocking other users from simply browsing the catalog.

5.2.5.2 Search Optimization: Testing the response time of the component search bar as the MySQL database scales to contain thousands of local inventory records.

5.3 Testing Strategies

While testing techniques define the specific tests conducted on the platform, the testing strategy defines the overarching methodology and sequence in which those tests are applied. Given the decoupled architecture of Tech Shopway—incorporating a React.js frontend, a Django backend, and external cloud AI services—a structured, phased approach was essential to isolate bugs efficiently.

The following primary testing strategies were implemented:

5.3.1 White Box Testing: White Box testing is an approach where the internal structure, design, and coding logic of the software are tested by developers who have full knowledge of the source code.

5.3.1.1 Backend Logic Verification: Developers directly examined the Python code to ensure the Django Object-Relational Mapping (ORM) queries were highly optimized and not causing "N+1 query" problems when pulling data from the multi-vendor junction tables.

5.3.1.2 Vector Search Accuracy: The internal logic of the Retrieval-Augmented Generation (RAG) pipeline was tested to ensure that text chunking and vector embeddings in ChromaDB accurately matched the semantic meaning of the component datasheets, rather than just testing the final output.

5.3.2 Black Box Testing (Behavioral Testing): Black Box testing focuses on the software's functionality without any knowledge of internal code structure. It tests the system purely on inputs and expected outputs, simulating how a real student or vendor would experience the platform.

5.3.2.1 UI/UX Validation: Testers interacted with the React.js frontend, attempting to input invalid characters into the Kit Builder, submitting incomplete vendor registration forms, and checking if the system returned the correct, user-friendly error messages without crashing.

5.3.2.2 API Response Validation: Sending generic RESTful HTTP requests (via tools like Postman) to the backend to ensure that providing a specific `ComponentID` consistently returned the correct localized price list, regardless of how the Python code achieved it.

5.3.3 Bottom-Up Integration Strategy: Because the system is highly modular, a "Bottom-Up" integration strategy was chosen. In this approach, testing begins at the lowest, most fundamental levels of the architecture before progressively integrating and testing higher-level modules.

5.3.3.1 Phase 1 (Database Layer): First, the MySQL schema and ChromaDB instances were tested to ensure data integrity and successful migrations.

5.3.3.2 Phase 2 (API Layer): Next, the Django REST Framework endpoints were tested to ensure they could successfully read and write to the verified databases.

5.3.3.3 Phase 3 (Client Layer): The React frontend was then connected to the API layer, testing the flow of data from the database to the user's screen.

5.3.3.4 Phase 4 (External AI Layer): Finally, the external Gemini API calls were integrated and tested, ensuring the core system remained stable even if the AI service experienced latency.

5.3.4 User Acceptance Testing (UAT): UAT is the final phase of the testing strategy, validating that the software not only works technically but actually solves the real-world business problems it was designed for.

5.3.4.1 Alpha Testing (Internal): The development team simulated both Vendor and Student roles, running complete end-to-end scenarios (e.g., creating a shop, listing an Arduino, and a student buying it for a project).

5.3.4.2 Beta Testing (Target Audience): A small group of engineering students from Dr. A.P.J. Abdul Kalam Technical University (AKTU) were given access to the platform. They were tasked with using the "Kit Builder" to design a theoretical 4th-year project. Their feedback was collected specifically on the accuracy of the AI compatibility reports and the ease of comparing local shop prices.

6. SYSTEM SECURITY MEASURES

6.1 Introduction

In modern web architecture, particularly for platforms handling multi-actor environments and proprietary business data, system security is not an optional add-on but a foundational requirement. The **Tech Shopway** platform manages sensitive information across three distinct user groups: engineering students (academic profiles, personal emails), local vendors (business credentials, proprietary pricing strategies), and system administrators.

Furthermore, the integration of external Artificial Intelligence engines (Google Gemini/Groq) introduces additional threat vectors regarding API abuse and data leakage. The primary objective of this chapter is to outline the comprehensive security protocols implemented within the decoupled React.js and Django architecture to protect data integrity, ensure high availability, and prevent unauthorized access.

6.2 Authentication and Authorization

Securing the platform begins with verifying who is accessing the system and strictly controlling what they are permitted to do once inside.

6.2.1 Role-Based Access Control (RBAC): Tech Shopway utilizes a strict RBAC matrix managed by the Django backend. The system recognizes distinct user roles (Student, Vendor, Admin). A Student account is programmatically restricted from accessing vendor inventory modification endpoints, and a Vendor cannot access the global administrative dashboard.

6.2.2 Secure Session Management: Instead of relying on basic authentication, the system uses JSON Web Tokens (JWT) or secure HTTP-only session cookies. Upon successful login, the server issues a token containing the user's role and ID. This token must be passed with every API request from the React frontend.

6.2.3 Password Cryptography: Plaintext passwords are never stored in the MySQL database. Django utilizes strong, one-way cryptographic hashing algorithms (such as PBKDF2 with a SHA256 hash or Argon2) alongside unique cryptographic "salts" for each user, rendering database breaches useless to attackers attempting to steal passwords.

6.3 Application-Level Threat Mitigation

The platform is designed to defend against the most common web application vulnerabilities, heavily leveraging the built-in security middleware provided by the Django framework.

6.3.1 Cross-Site Scripting (XSS) Prevention: XSS occurs when an attacker injects malicious scripts into web pages viewed by other users. Tech Shopway prevents this by automatically escaping all user-submitted data before it is rendered on the React frontend. For example, if a vendor attempts to put a JavaScript payload in a component description, the system treats it strictly as plain text.

6.3.2 SQL Injection (SQLi) Protection: Because the backend relies on Django's Object-Relational Mapper (ORM), raw SQL queries are rarely executed. The ORM automatically parameterizes all database queries, completely neutralizing attempts by attackers to manipulate the MySQL database through malicious search bar inputs.

6.3.3 Cross-Site Request Forgery (CSRF) Defense: To prevent attackers from tricking authenticated users into executing unwanted actions (like deleting a project kit), the backend enforces CSRF token validation on all state-changing requests (POST, PUT, DELETE).

6.4 API and Infrastructure Security

Because Tech Shopway offloads heavy computational tasks to external cloud services, securing the infrastructure connecting these services is paramount.

6.4.1 API Key Management: The cryptographic keys required to access the Google Gemini and Groq APIs are highly sensitive. These keys are never hardcoded into the source code or committed to version control (GitHub). Instead, they are securely managed as isolated Environment Variables (`Confidential` files).

6.4.2 Rate Limiting and Throttling: To protect the platform from Denial of Service (DoS) attacks and to prevent abuse of the paid external AI APIs, rate limiting is enforced at the server level. The system restricts the number of "AI Feasibility Analysis" requests a single IP address or user account can make within a specific timeframe.

6.4.3 Data in Transit Encryption (TLS/SSL): All communication between the student's browser, the local vendor's dashboard, the Django server, and the external AI APIs is encrypted using Transport Layer Security (HTTPS). This prevents "man-in-the-middle" attackers from intercepting user data or API payloads over unsecure networks.

7. SYSTEM MODULES/REPORTS AND THEIR DESCRIPTIONS

7.1 Module 1: Login / Registration (Identity & Access Management)

This module serves as the secure gateway to the platform, handling user onboarding, authentication, and strict role separation.

7.1.1 Target Users: Students, Local Vendors, and System Administrators.

7.1.2 Key Features:

7.1.2.1 Role-Specific Onboarding: Distinct registration flows for engineering students and local shop owners.

7.1.2.2 Secure Authentication: Utilizes cryptographic hashing for passwords and issues secure JSON Web Tokens (JWT) for session management.

7.1.2.3 State Initialization: Upon login, this module dictates which subsequent modules (like the Seller Hub) become visible based on the user's validated role.

7.2 Module 2: Project Analysis

This module provides automated technical validation for hardware projects, ensuring that selected components are theoretically compatible before physical assembly.

7.2.1 Target Users: Engineering Students.

7.2.2 Key Features:

7.2.2.1 Automated Validation: Cross-references the voltage logic levels, communication protocols (I2C, SPI, UART), and power requirements of the components in a student's kit.

7.2.2.2 BOM Generation: Automatically generates a formatted Bill of Materials (BOM) for college project documentation.

7.2.2.3 Error Highlighting: Flags critical hardware mismatches (e.g., connecting a 5V sensor directly to a 3.3V microcontroller pin without a logic converter).

7.3 Module 3: AI Lab Assistant

This module houses the platform's advanced conversational AI, powered by the Retrieval-Augmented Generation (RAG) pipeline and external Large Language Models (Gemini/Groq).

7.3.1 Target Users: Engineering Students.

7.3.2 Key Features:

7.3.2.1 Context-Aware Q&A: Students can ask complex technical questions about their specific hardware.

7.3.2.2 Vector Database Integration: The assistant queries the local ChromaDB to pull exact technical specifications from component datasheets before generating an answer.

7.3.2.3 Code Generation: Can assist students by generating boilerplate C++/Python code for the specific microcontrollers and sensors they have selected.

7.4 Module 4: Smart Kit Builder

This is the primary interactive workspace where students design their hardware projects.

7.4.1 Target Users: Engineering Students.

7.4.2 Key Features:

7.4.2.1 Dynamic Workspace: A React-based interface allowing students to virtually assemble, configure, and save different component kits.

7.4.2.2 Real-time State Management: Instantly updates total project costs and component lists as items are added or removed, without requiring page reloads.

7.4.2.3 Persistent Storage: Saves the virtual kits to the student's database profile for future access and editing.

7.5 Module 5: Product Explorer

This module acts as the global search engine for the platform's entire hardware catalog.

7.5.1 Target Users: Engineering Students and Vendors (for market research).

7.5.2 Key Features:

7.5.2.1 Advanced Filtering: Allows users to filter components by category (Microcontrollers, Actuators, Passives), technical specifications, and availability.

7.5.2.2 Centralized Data: Displays the master data for a component (e.g., generic ESP32 specs) before breaking down into vendor-specific listings.

7.6 Module 6: Price Intelligence

This module is the core of the "Aggregator" functionality, designed to optimize student budgets.

7.6.1 Target Users: Engineering Students.

7.6.2 Key Features:

7.6.2.1 Cross-Vendor Comparison: Takes a specific component and displays a side-by-side comparison of prices and stock levels from all local shops carrying that item.

7.6.2.2 Budget Optimization: Suggests the most cost-effective combination of vendors to purchase a complete kit from, minimizing the overall financial cost for the student.

7.7 Module 7: Project Ideas

This module serves as an inspiration hub and a starting point for students who are unsure where to begin.

7.7.1 Target Users: Junior Engineering Students / Beginners.

7.7.2 Key Features:

7.7.2.1 Template Library: Hosts a repository of pre-configured, standard college projects (e.g., "Line Follower Robot," "Weather Station," "Smart Dustbin").

7.7.2.2 One-Click Import: Allows students to clone a project idea directly into their Smart Kit Builder, instantly populating their workspace with the required components.

7.8 Module 8: Local Shops

This module functions as a localized business directory, bridging the gap between the college campus and the surrounding electronic markets.

7.8.1 Target Users: Engineering Students.

7.8.2 Key Features:

7.8.2.1 Vendor Profiles: Displays shop names, contact information, operational hours, and physical addresses.

7.8.2.2 Shop-Specific Catalogs: Allows a student to browse the entire available inventory of one specific local vendor, rather than searching globally.

7.9 Module 9: Seller Hub

This is a highly restricted, secure backend-facing module designed exclusively for shop owners to manage their business on the platform.

7.9.1 Target Users: Local Vendors (Strictly Restricted Access).

7.9.2 Key Features:

7.9.2.1 Inventory Management (CRUD): Vendors can add new components to their local catalog, update stock status, and remove discontinued items.

7.9.2.2 Dynamic Pricing Engine: Allows vendors to instantly update their localized pricing, which pushes immediate updates to the Product Explorer and Price Intelligence modules.

7.9.2.3 Business Analytics: A dashboard displaying metrics such as which of their components are most frequently viewed or added to student kits.

8. FUTURE SCOPE OF THE PROJECT

Community Discussion Hub: Integration of a dedicated forum where students can collaborate, troubleshoot complex hardware issues, request component alternatives, and share domain knowledge directly within the platform.

Community Market Place (Quikr/OLX model): Development of a localized, student-to-student classifieds system. This will allow senior students to sell or trade gently used microcontrollers and sensors to juniors, promoting hardware upcycling and further reducing project costs.

Mobile App: Creation of a native cross-platform mobile application (utilizing frameworks like React Native or Flutter) to provide students with on-the-go access to their virtual workspaces and real-time push notifications for vendor stock updates.

Upload Project Details: Implementation of an open-source repository feature where students can formally publish their completed projects—including validated Bills of Materials (BOMs), code snippets, and final reports—serving as a reference library for future students.

Enhance Kit Builder into a Proper Workspace (with Collaboration): Upgrading the existing Kit Builder from a single-user tool into a synchronous, multi-player workspace (similar to Google Docs), enabling project groups to collaboratively design architectures and modify component lists in real-time.

Focus on Software Development Tools (for Non-CS/Non-Tech Students): Expanding the platform's interdisciplinary utility by providing tailored software resources, low-code integrations, and simplified logic pathways specifically designed to help non-Computer Science students successfully build IoT and hardware projects.

Expand AI Agent to Agentic AI (with Workspace Integration): Evolving the current AI Lab Assistant from a reactive checker into a proactive "Agentic AI." This AI will be deeply embedded in the workspace, capable of autonomously suggesting missing components, drafting starter code, and warning users of power bottlenecks as they build. Also generate circuit diagrams automatically based on the microcontrollers and sensors present in a user's Kit.

Direct Integration with College Labs: Partnering with university administrations to link the platform with internal college electronics lab inventories, allowing students to check for and reserve free, borrowable parts before purchasing from local vendors.

Voice Command Integration: Implementing Natural Language Processing (NLP) voice-to-text functionality, allowing students to interact with the AI and modify their virtual workspace hands-free while they are physically soldering or assembling hardware on their benches.

Separate Chatbot for Support System: Deploying an isolated, specifically trained AI chatbot dedicated purely to platform navigation, user support, and vendor onboarding, keeping it entirely distinct from the core hardware feasibility AI.

9. APPENDICES

9.1 SCREENSHOTS AND THEIR DESCRIPTIONS

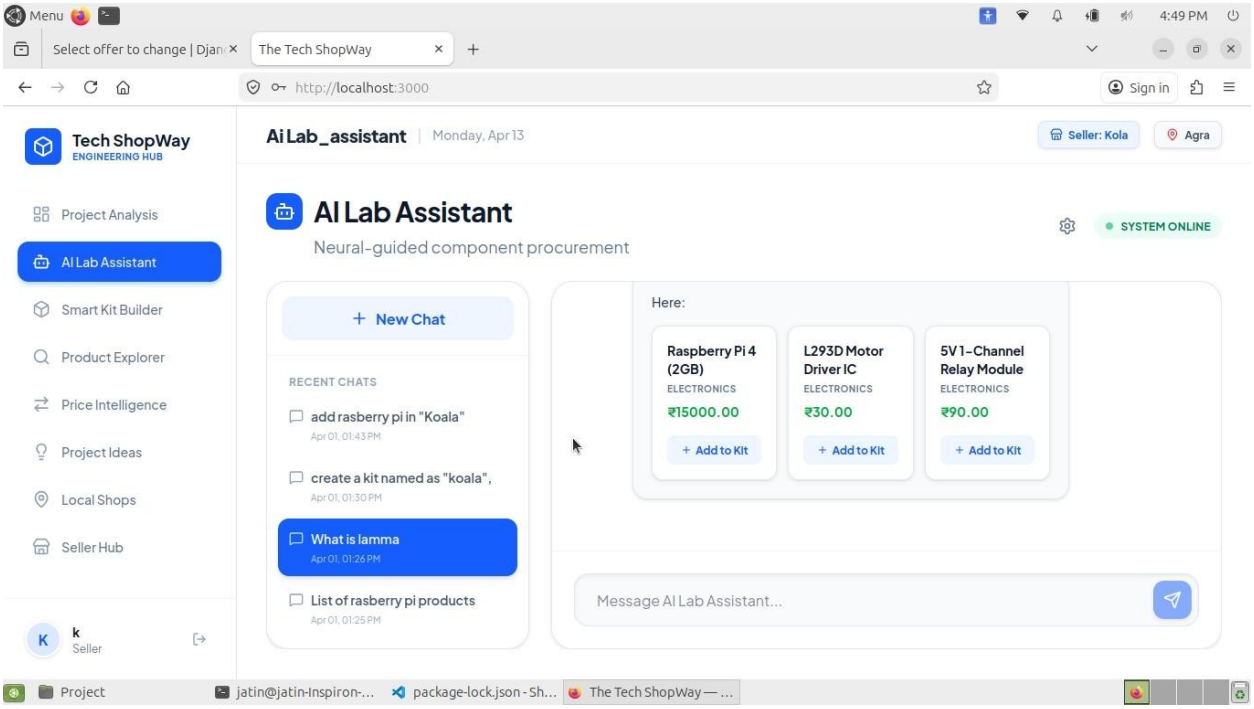

Image 9.1.1 AI Lab Assistant

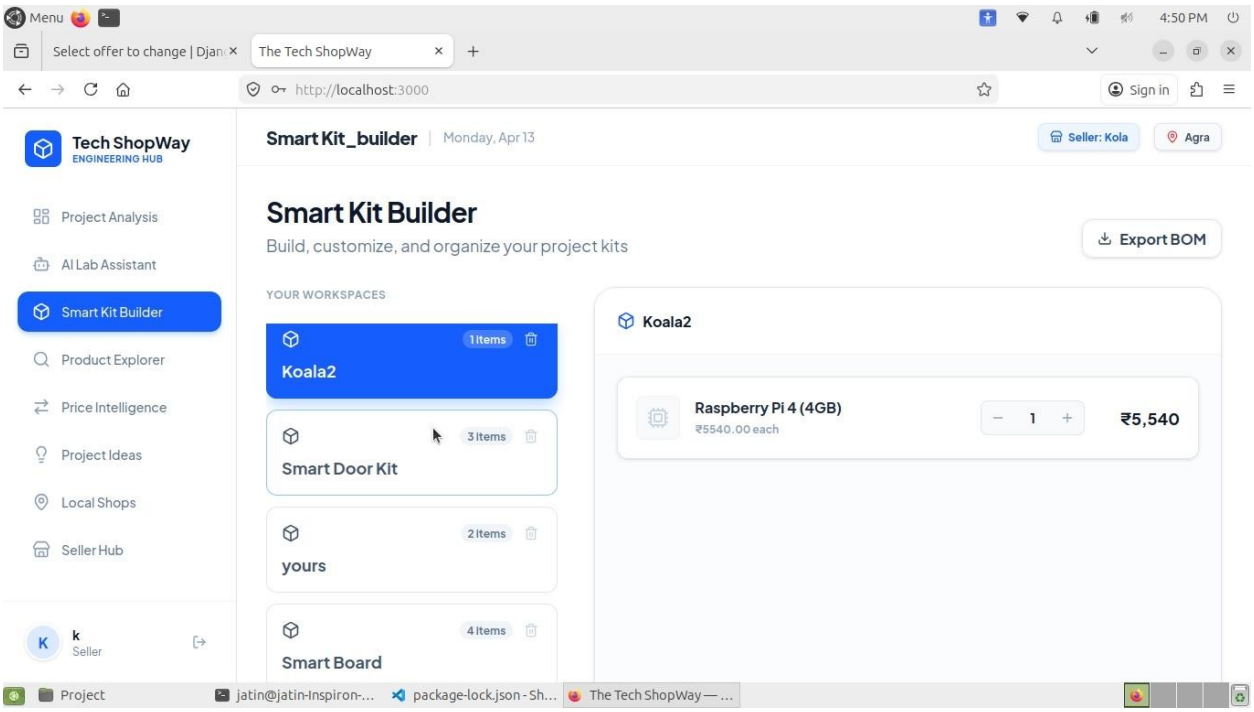

Image 9.1.2 Smart Kit Builder

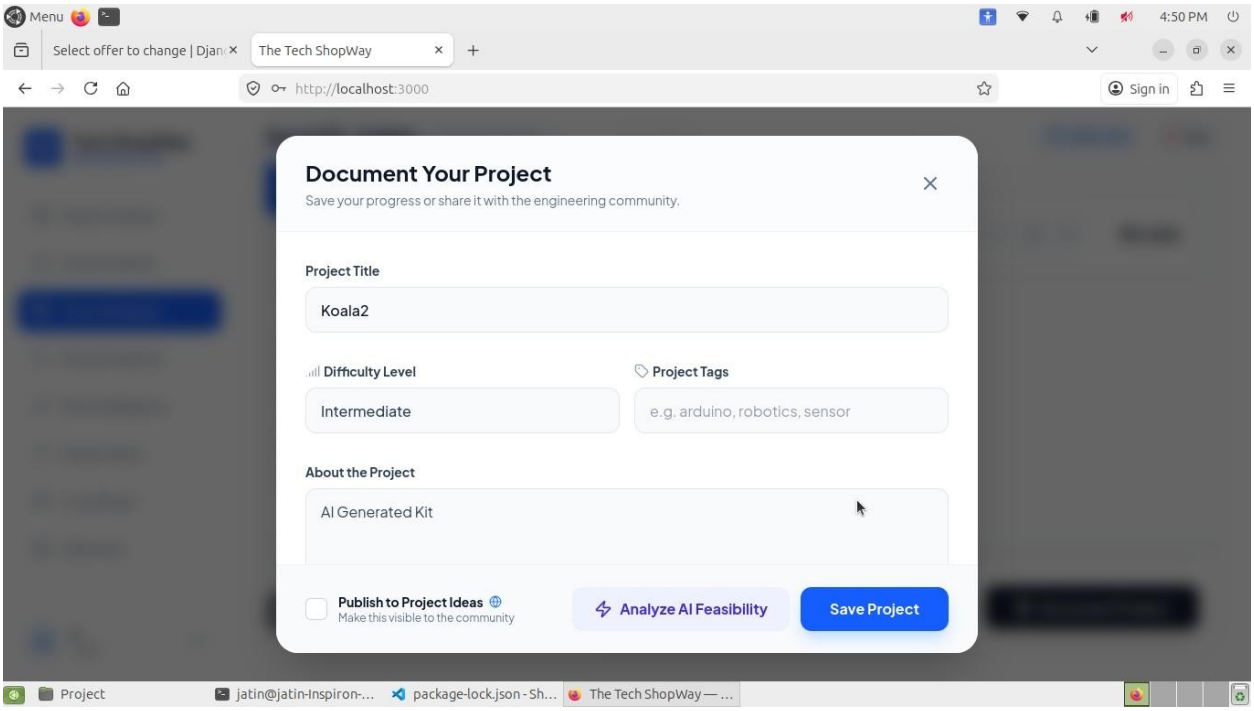

Image 9.1.3 Kit’s Document

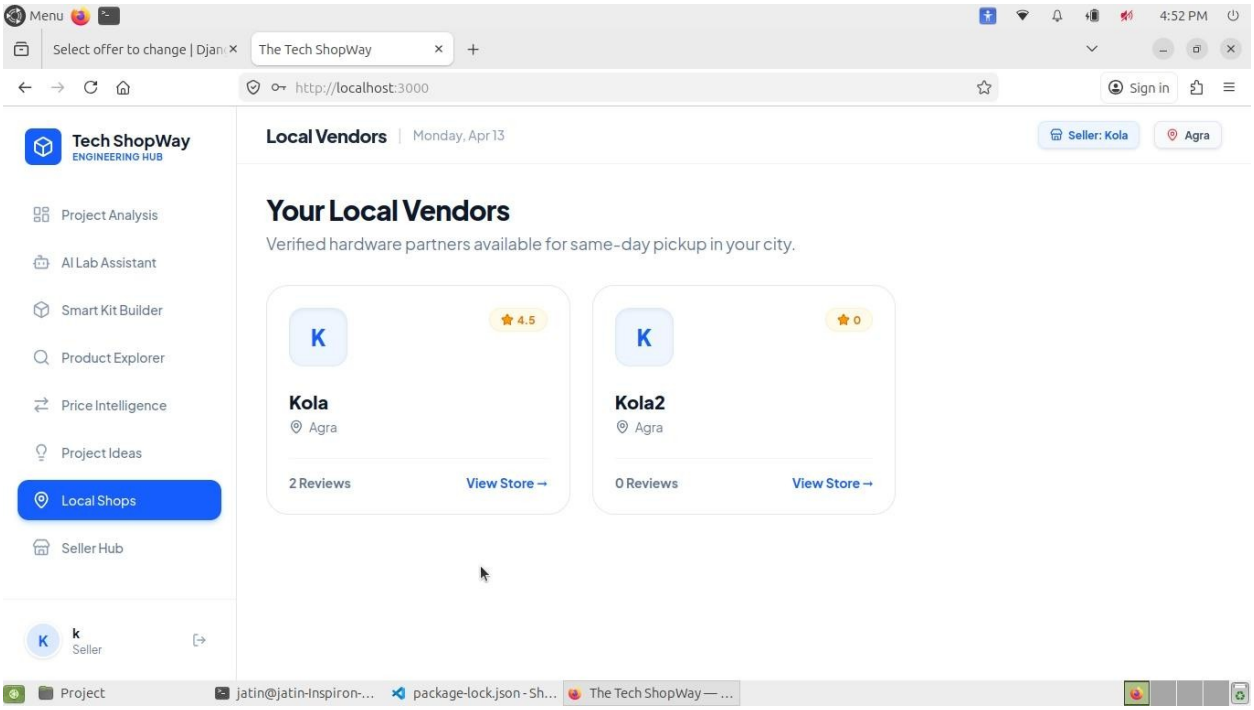

Image 9.1.4 Local Vendors

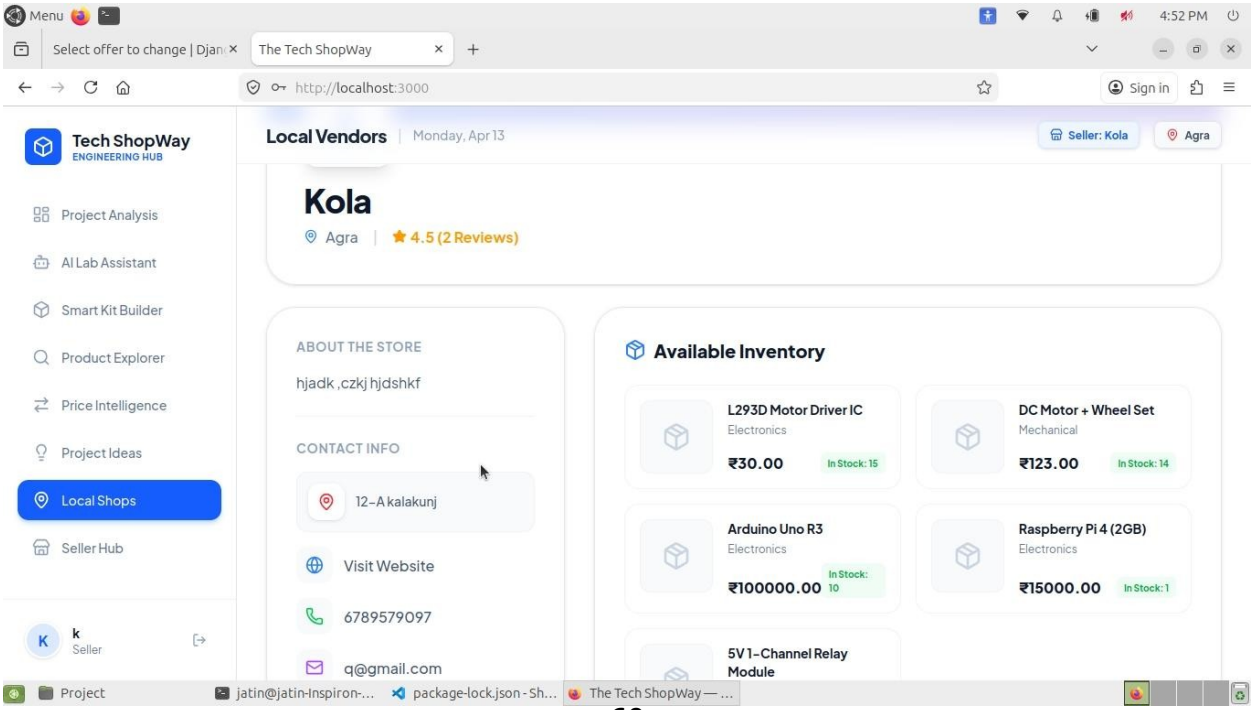

Image 9.1.5 A Local Shop’s description

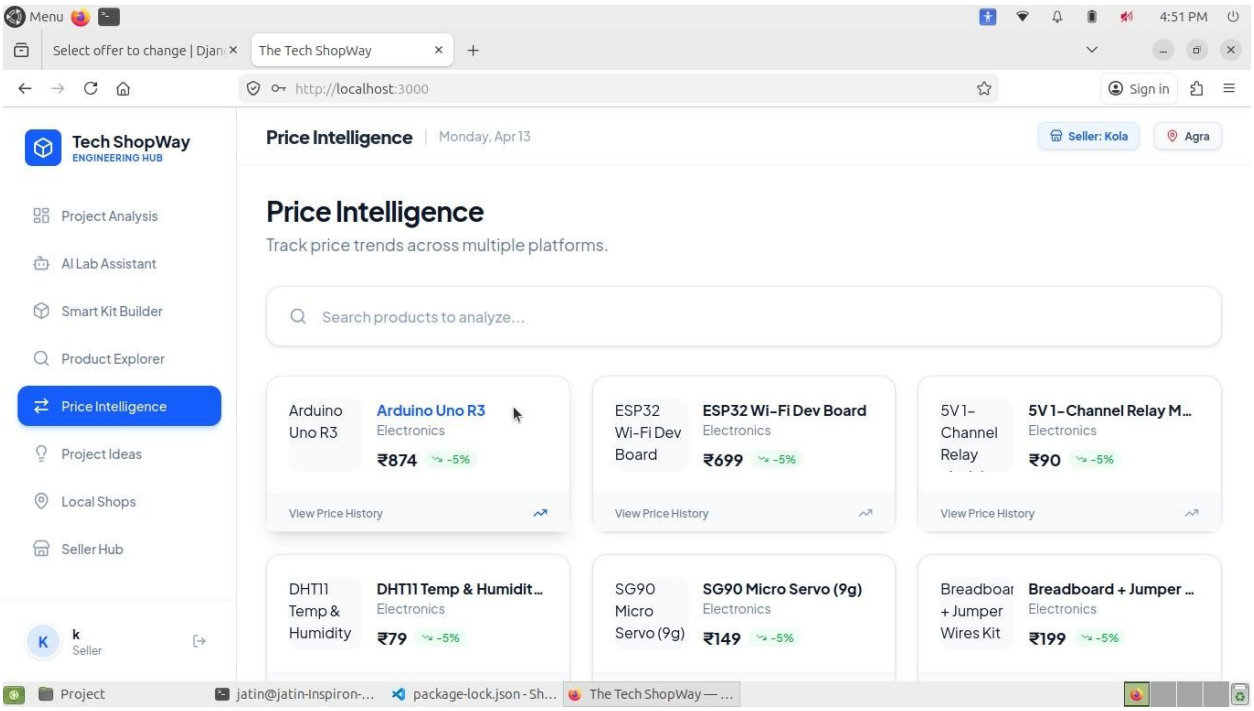

Image 9.1.6 Price History Section

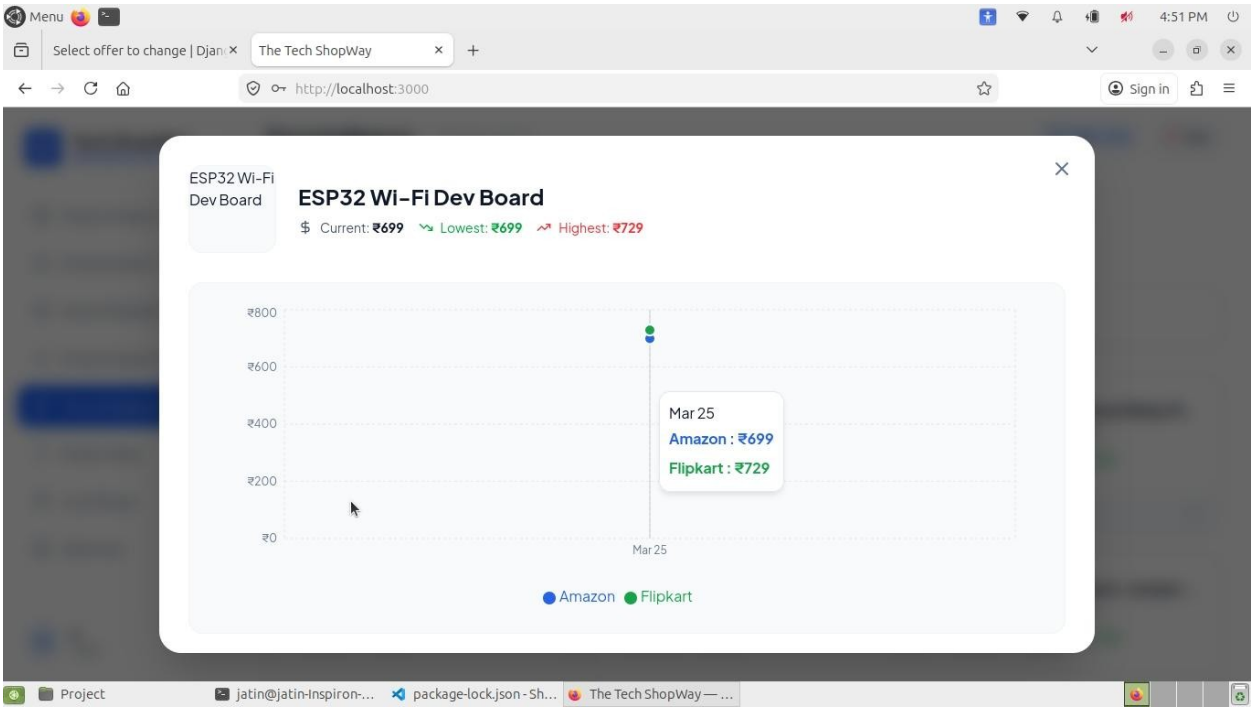

Image 9.1.7 Price History of a Product

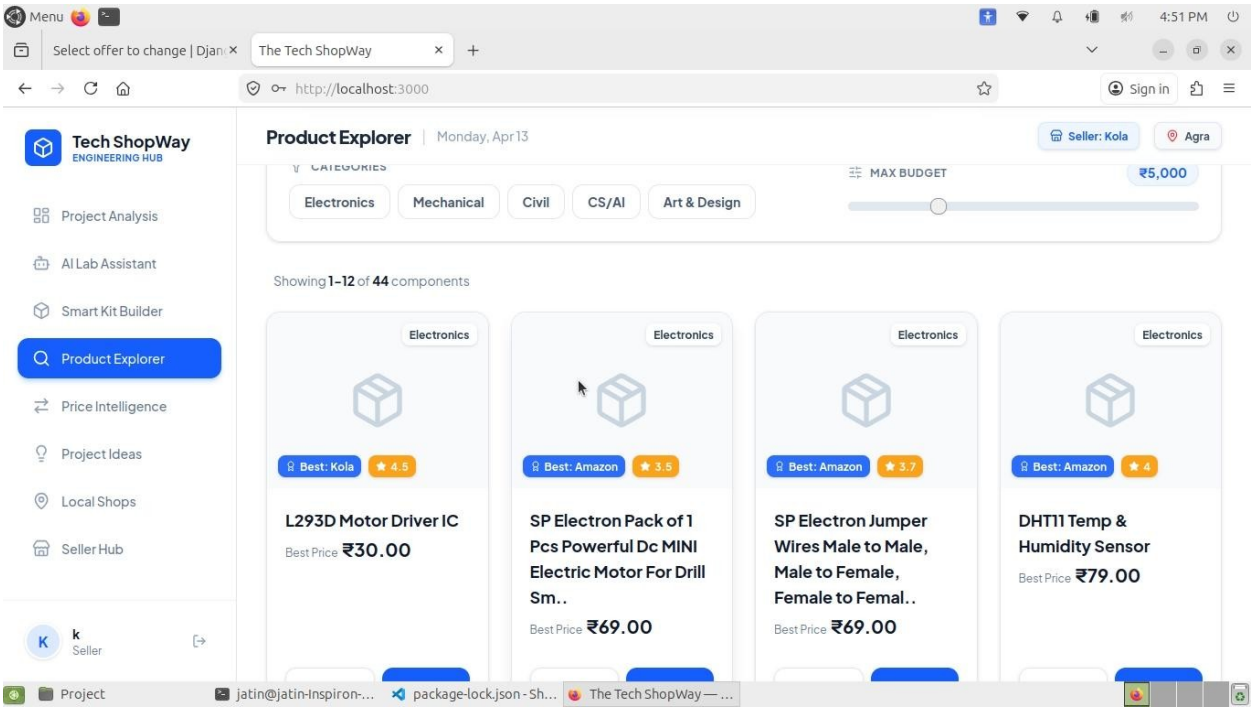

Image 9.1.8 Products Catalogue

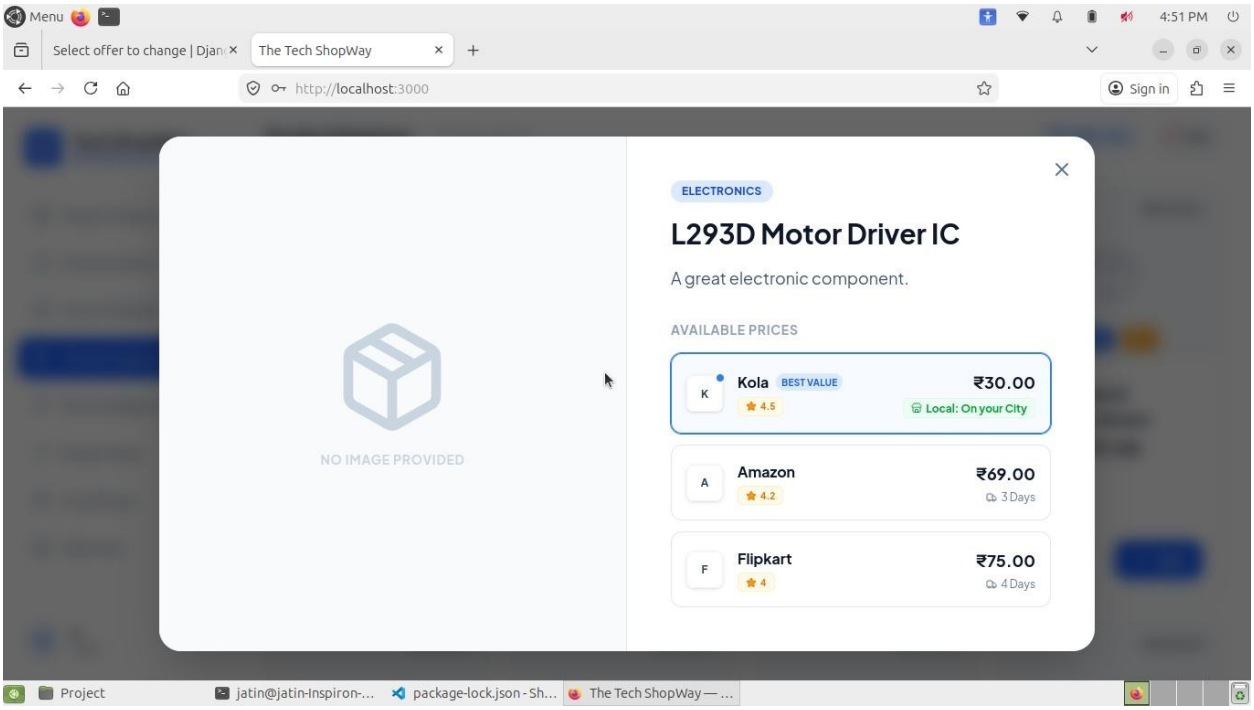

Image 9.1.9 Vendors for that Product

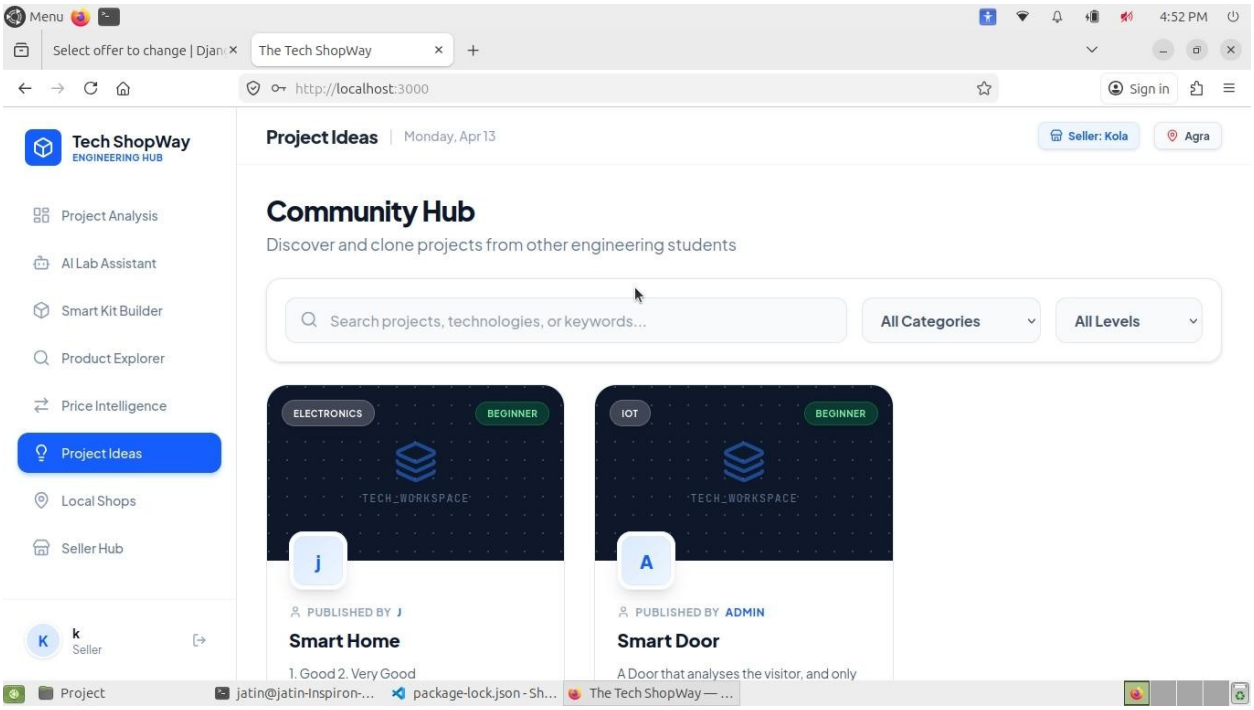

Image 9.1.10 Project Ideas of Different users

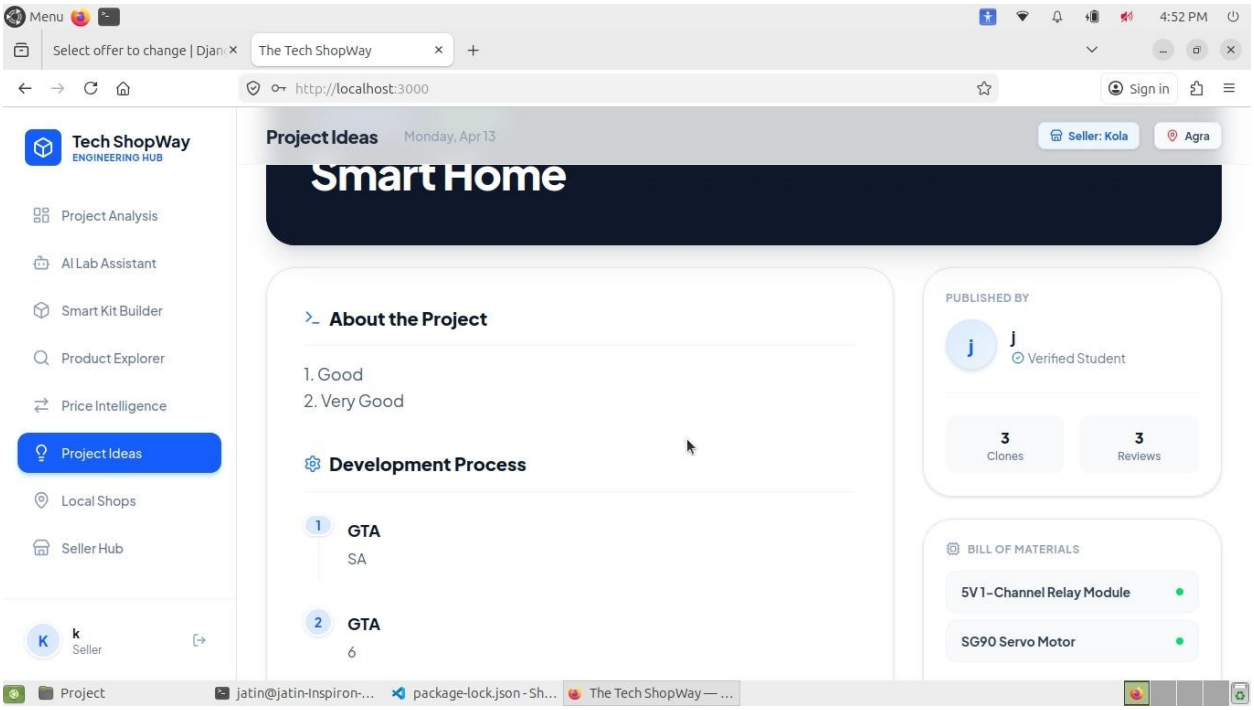

Image 9.1.11 An Idea’s Description

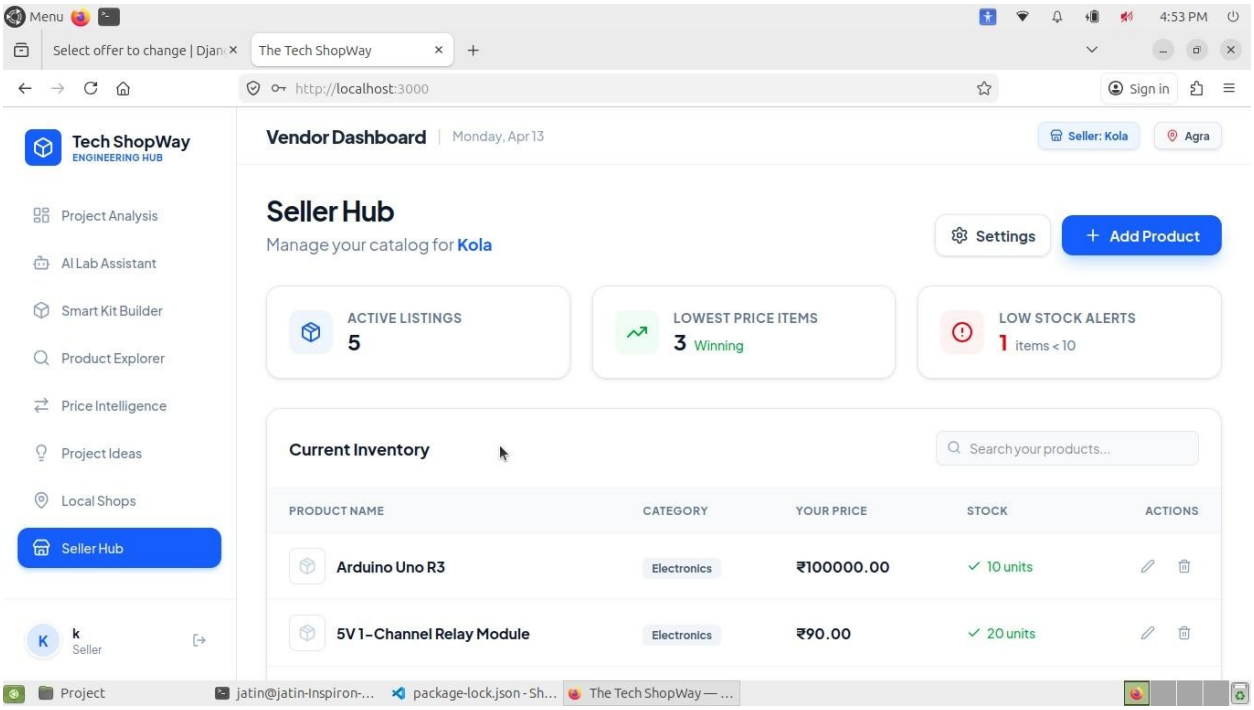

Image 9.1.12 Inventory Management for Vendors

10. BIBLIOGRAPHY/REFERENCES

1. Django Software Foundation. (2025). *Django Documentation*.
2. React. (2025). *React Documentation*. Meta Platforms, Inc.
3. Docker Inc. (2025). *Docker Compose and Containerization*.
4. Google. (2025). *Google Generative AI SDK Documentation*.
5. MySQL. (2025). *MySQL 8.0 Reference Manual*. Oracle Corporation.
6. Chroma. (2025). *ChromaDB Vector Database Documentation*.